

A decorative graphic on the left side of the slide, consisting of a network of white lines and small circles on a dark blue background, resembling a circuit board or a graph structure.

BI-PARTITE MATCHING AND REDUCTIONS

DATA STRUCTURES AND ALGORITHMS 2
MARK FLORYAN AND AARON BLOOMFIELD

ROADMAP: WHERE WE'RE GOING AND WHY

In Computer Science, we can often solve a new problem by casting it as an instance of another problem we already know how to solve. This is called a reduction.

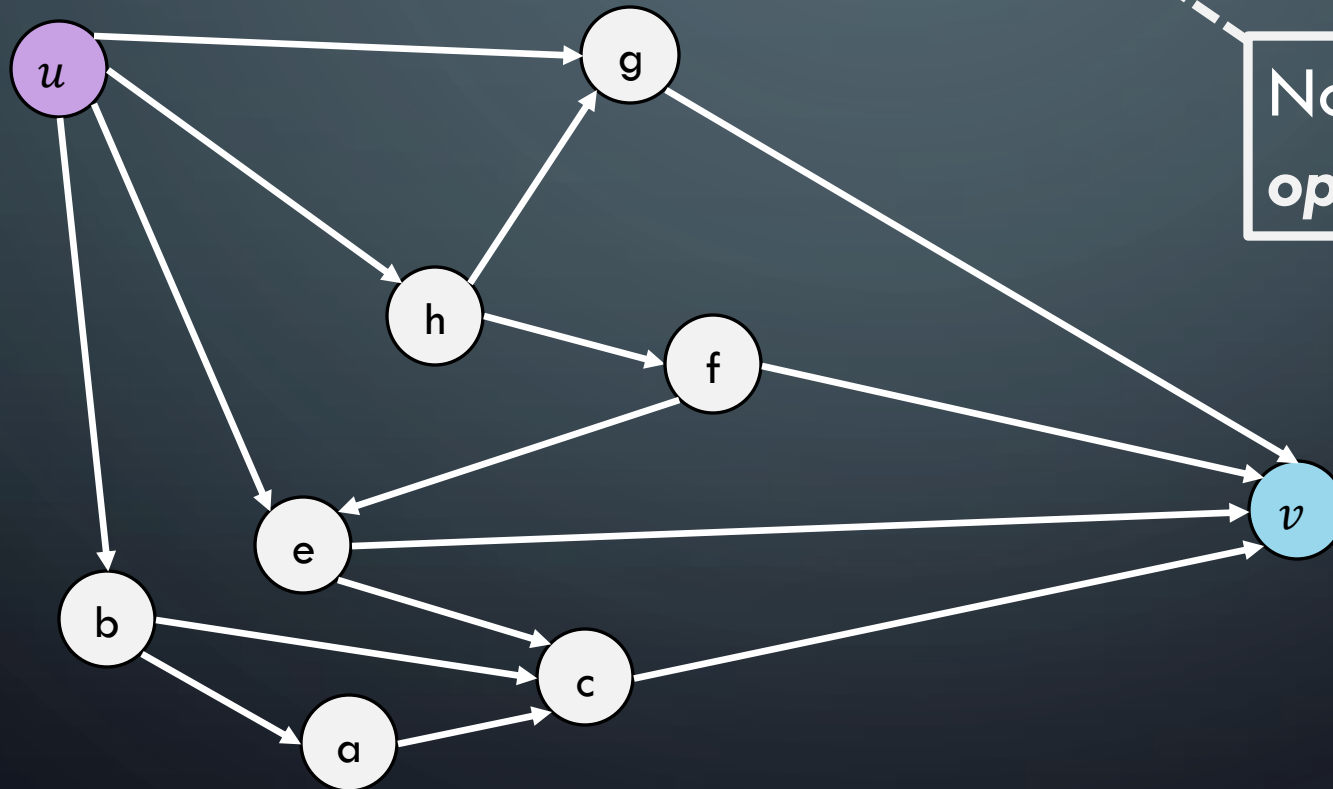
Let's see some examples, and then we will formalize this concept a bit more

The background is a dark blue gradient with a large, faint, light blue circle in the center. In the four corners, there are decorative white line art elements resembling circuit traces or branching paths, some ending in small circles.

EDGE DISJOINT PATHS

EDGE-DISJOINT PATHS

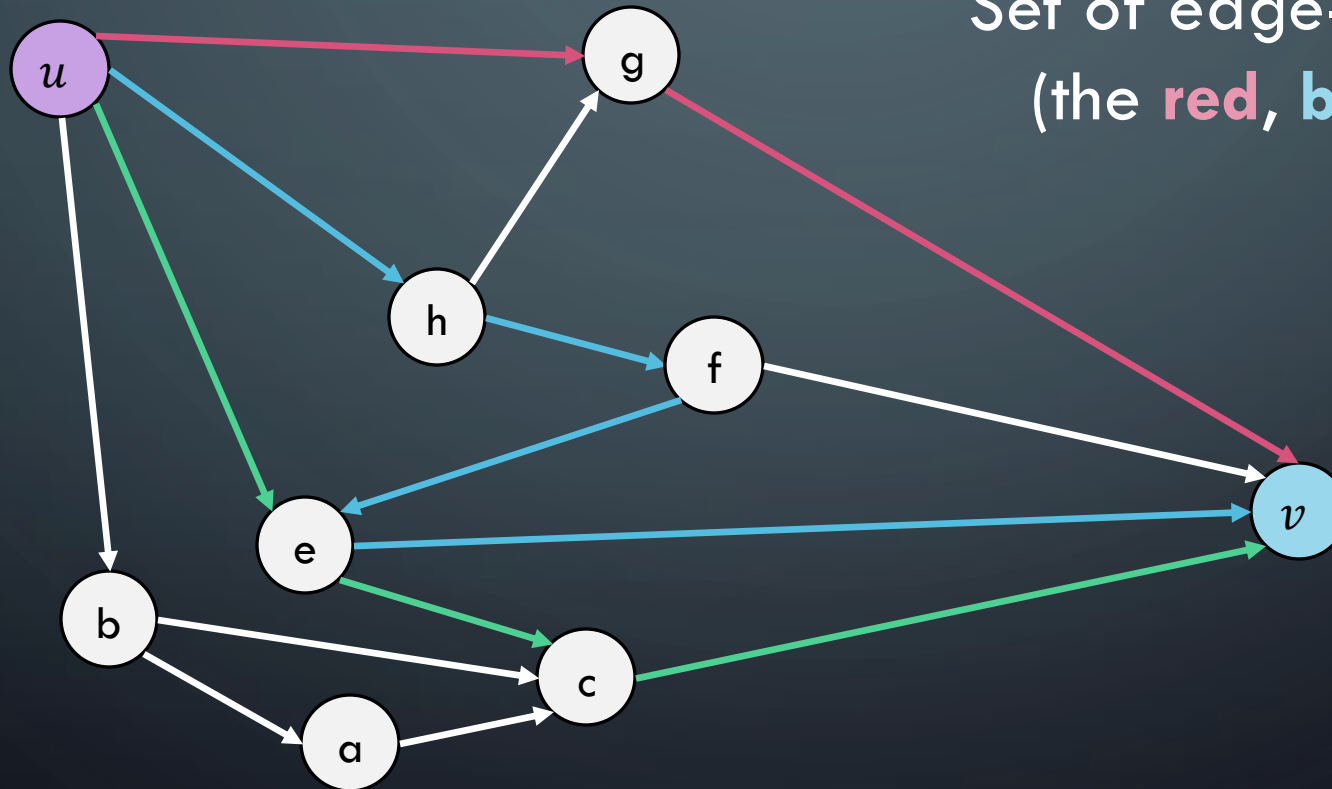
Given a graph $G = (V, E)$, a start node u and a destination node v , give the maximum number of paths from u to v which share no edges



Note this is an
optimization problem.

EDGE-DISJOINT PATHS

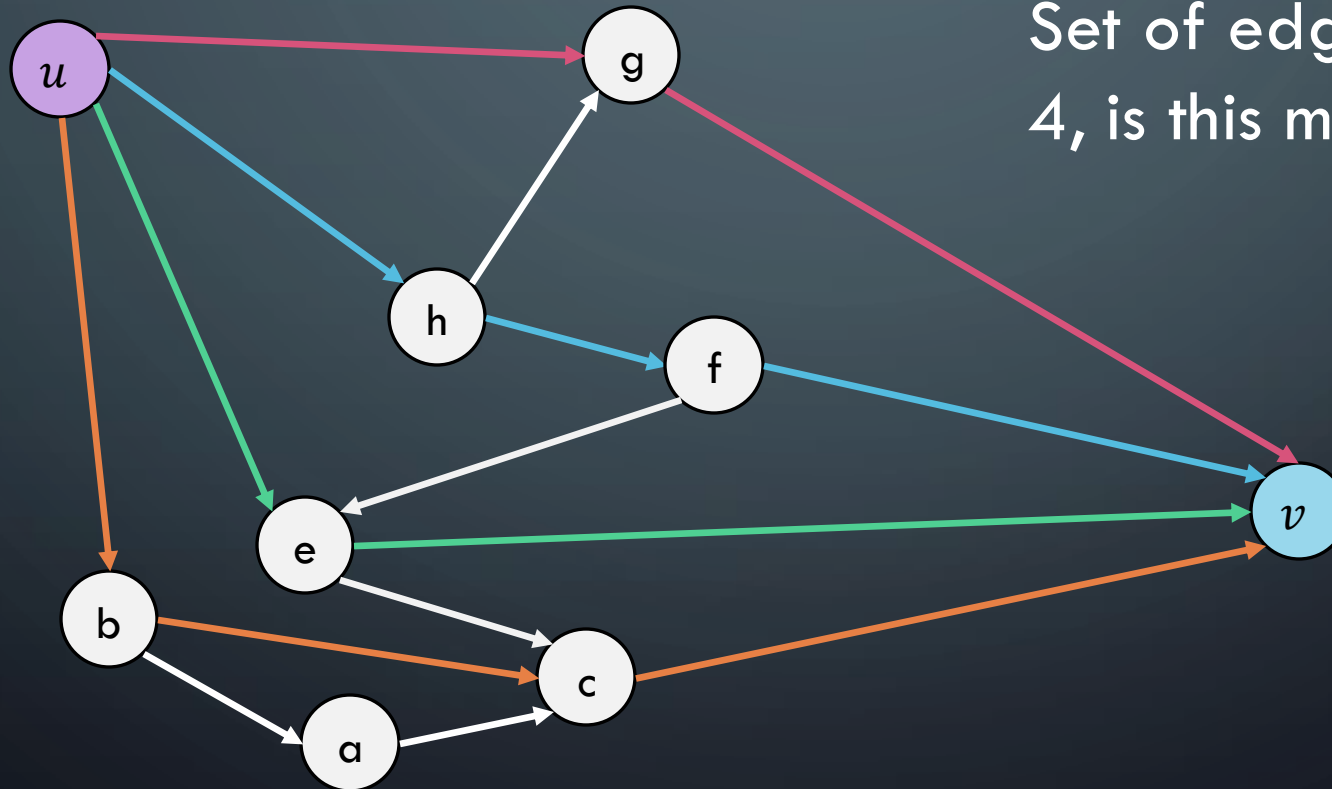
Given a graph $G = (V, E)$, a start node u and a destination node v , give the maximum number of paths from u to v which share no edges



Set of edge-disjoint paths of size 3
(the **red**, **blue**, **green** paths)

EDGE-DISJOINT PATHS

Given a graph $G = (V, E)$, a start node u and a destination node v , give the maximum number of paths from u to v which share no edges

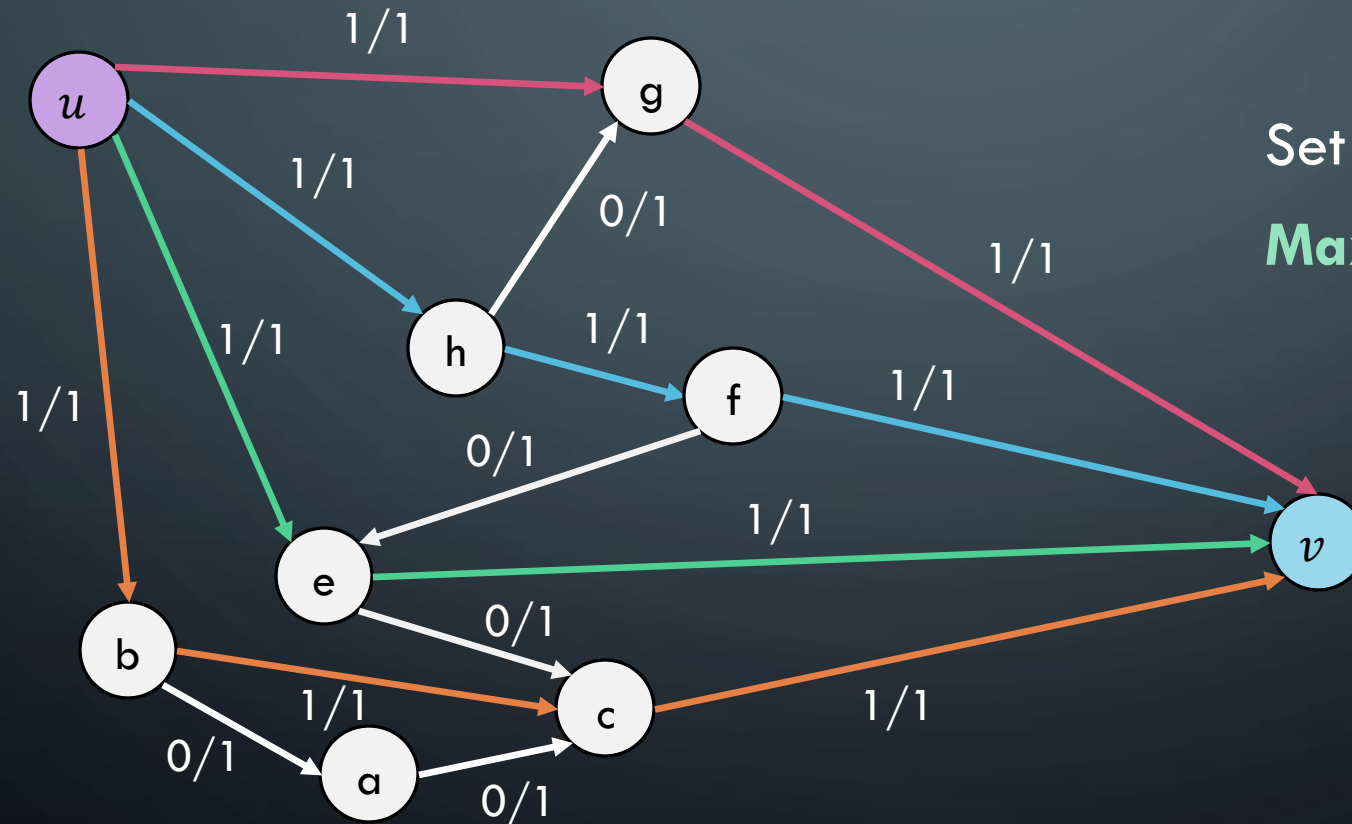


Set of edge-disjoint paths of size 4, is this max? Why or why not?

EDGE-DISJOINT PATHS ALGORITHM

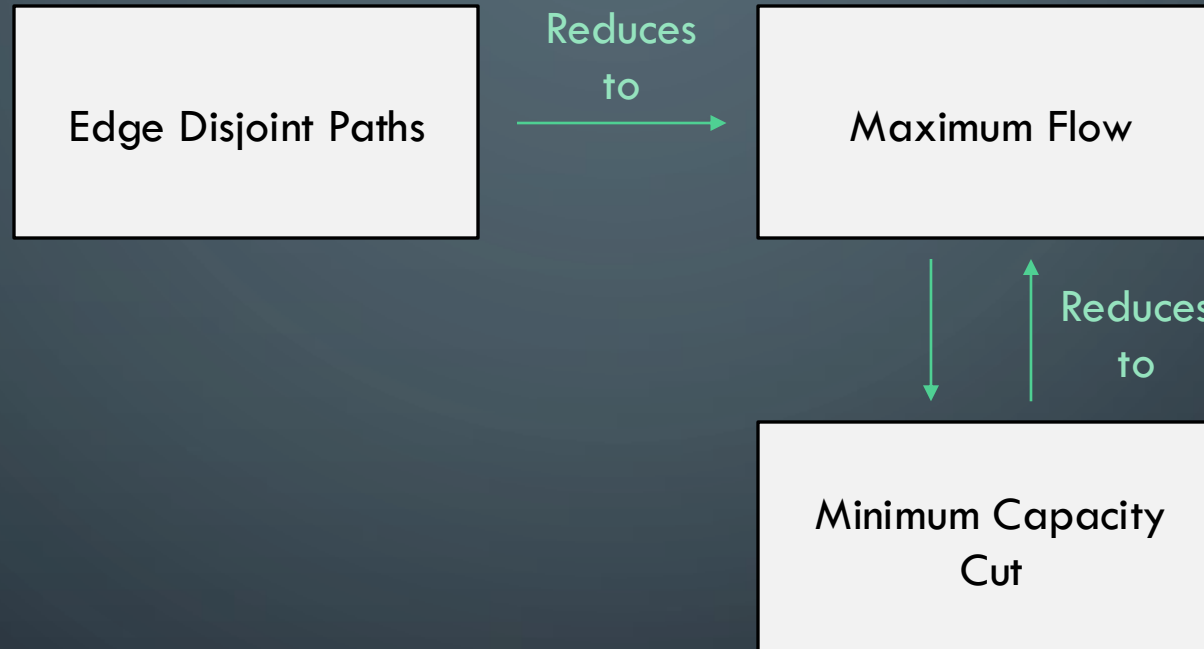
Use a problem we know how to solve, **max network flow**, to solve this!

Make u and v the source and sink, give each edge capacity 1, find the max flow.



Why does this work?
We need to be able to
make a valid argument
that it always does.

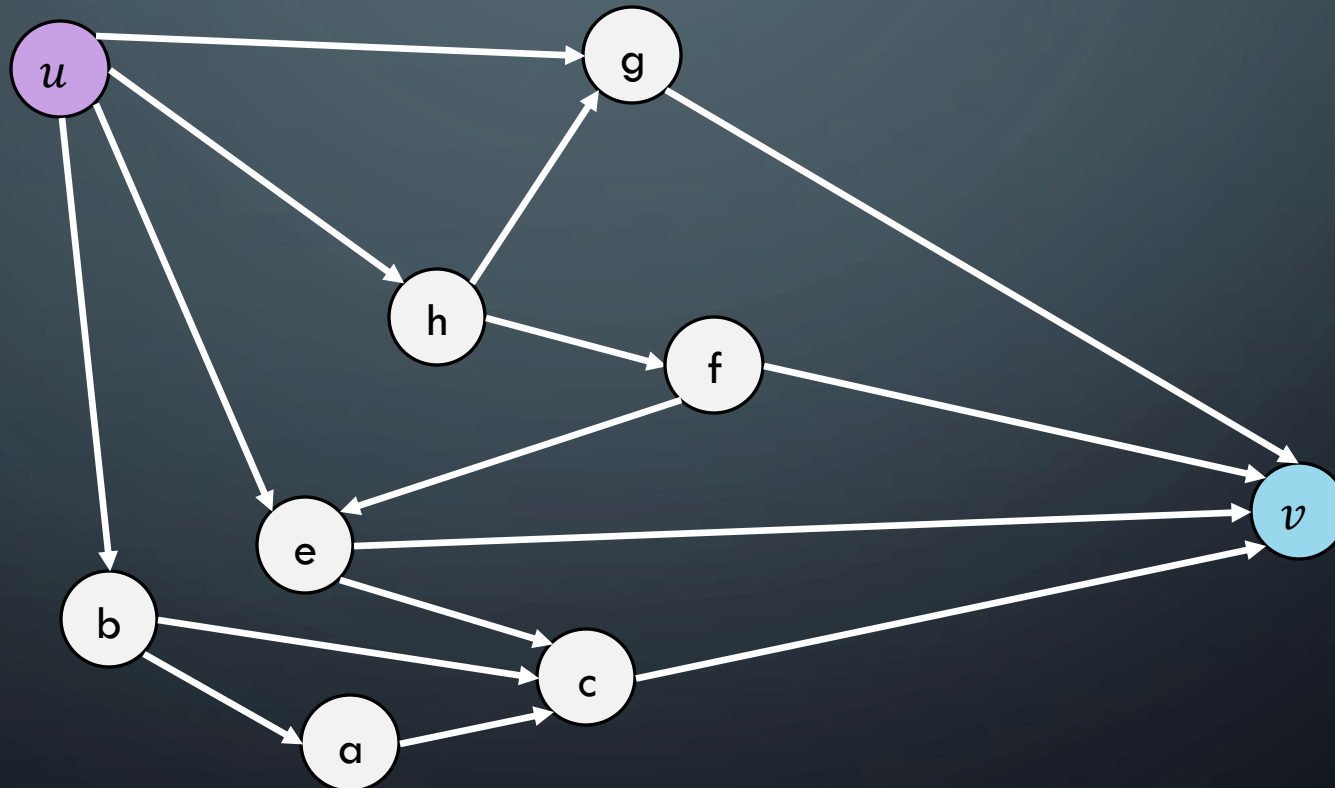
WHAT'S THE SITUATION?



EDGE DISJOINT PATHS

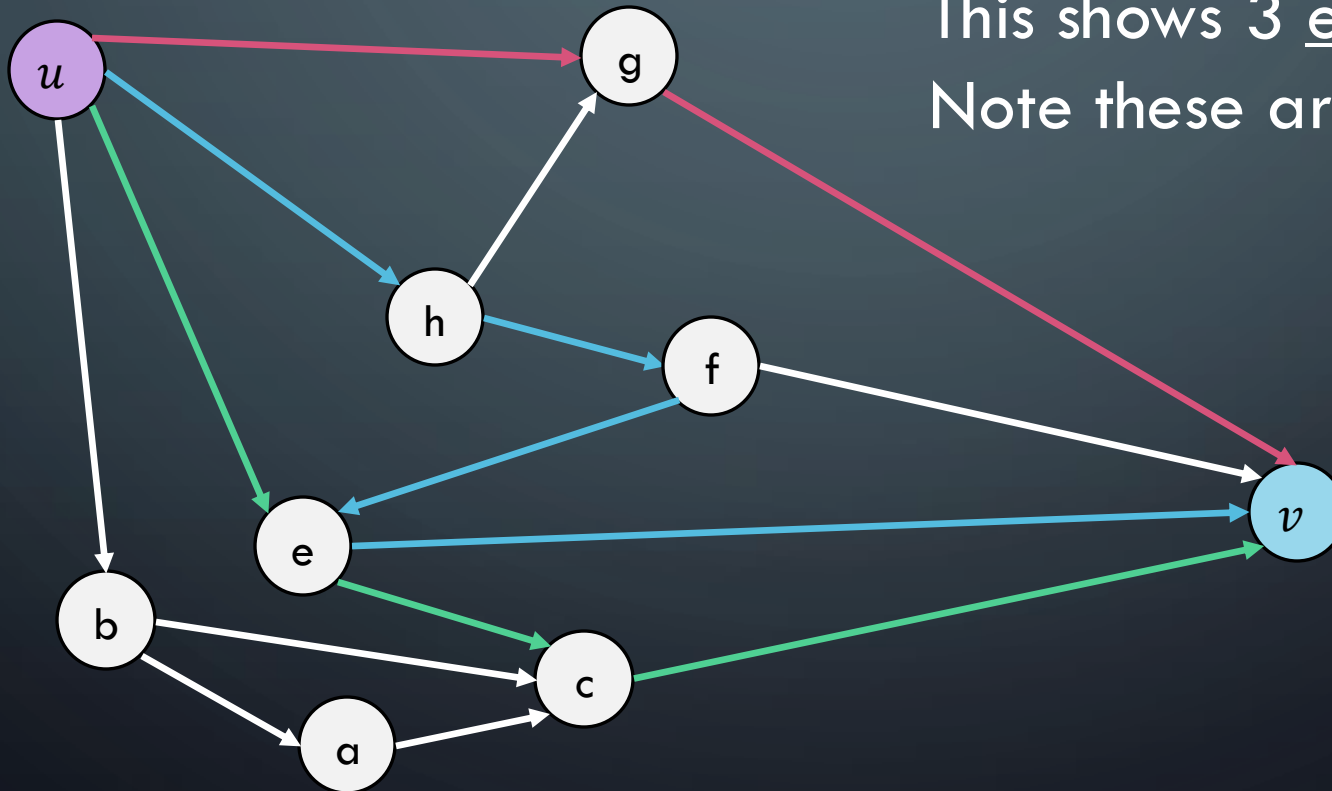
VERTEX-DISJOINT PATHS

Given a graph $G = (V, E)$, a start node u and a destination node v , give the maximum number of paths from u to v which share no vertices



VERTEX-DISJOINT PATHS

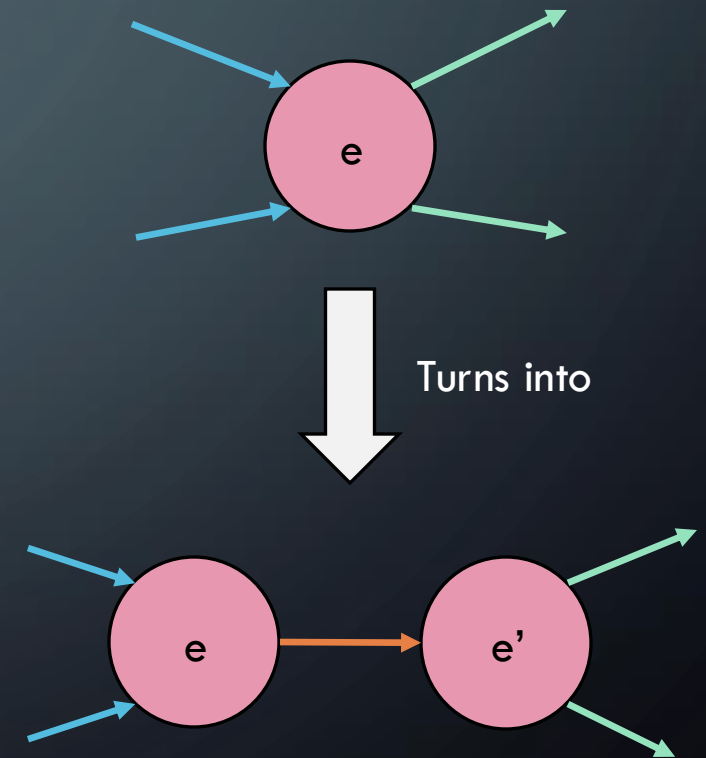
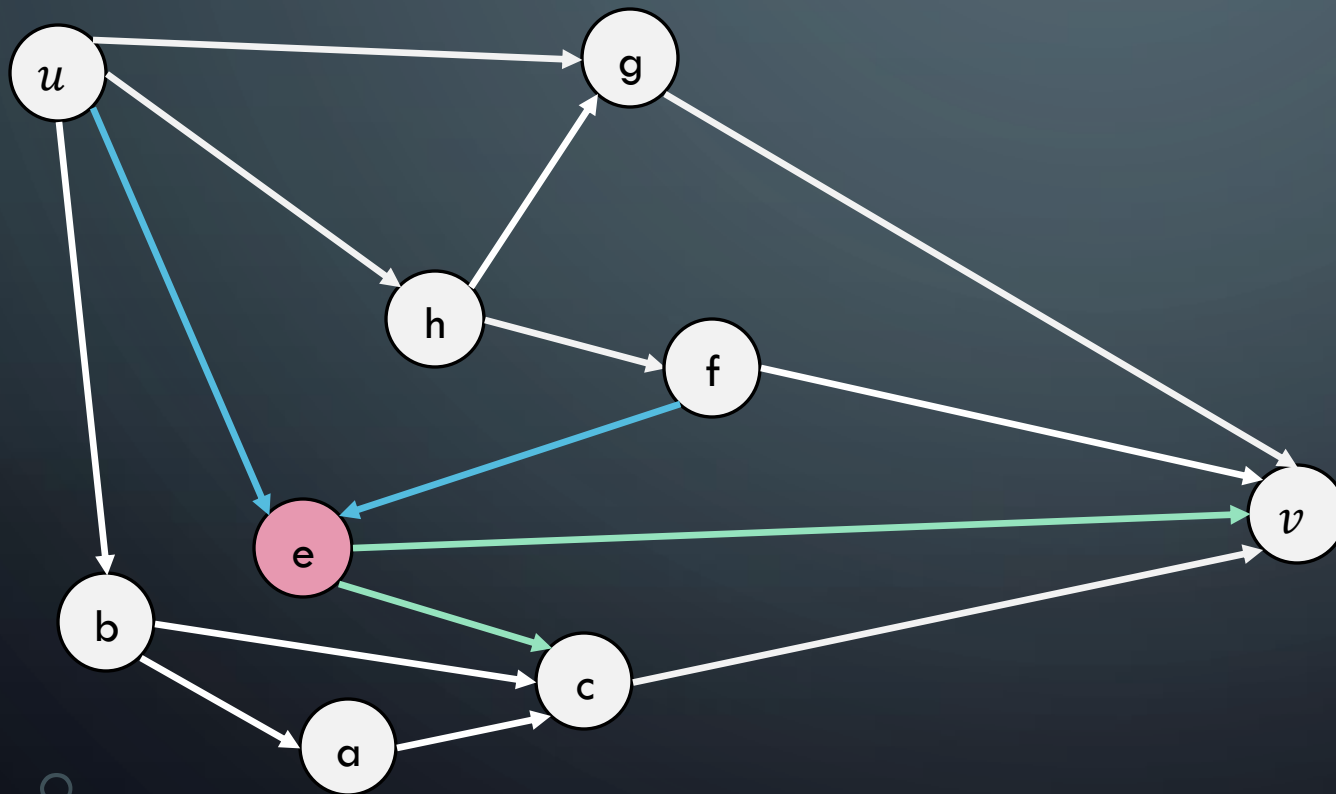
Given a graph $G = (V, E)$, a start node u and a destination node v , give the maximum number of paths from u to v which share no vertices



This shows 3 edge-disjoint paths.
Note these aren't vertex-disjoint paths!

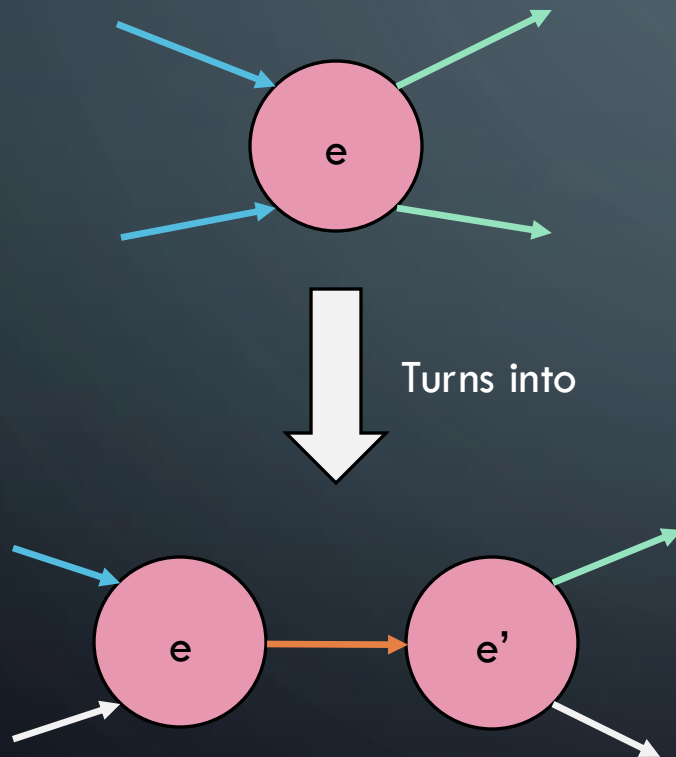
VERTEX-DISJOINT PATHS

Idea: Take each node and restrict how much many edge-disjoint paths can go through it. Split each node into two as shown.



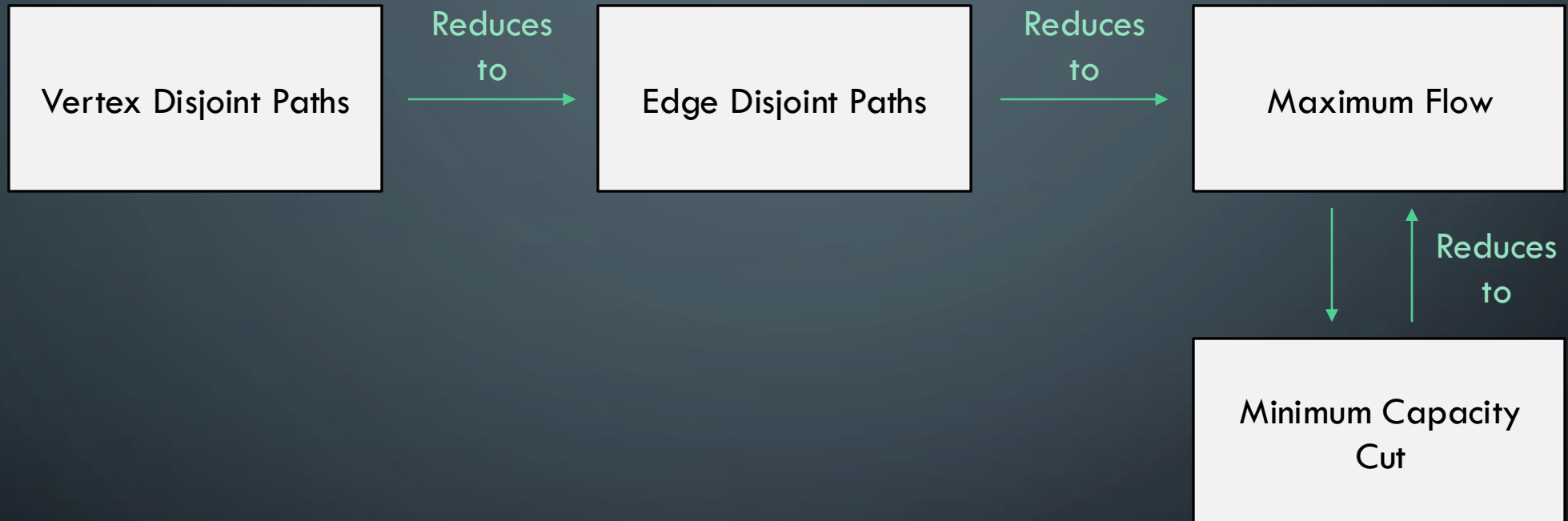
VERTEX-DISJOINT PATHS

Idea: Take each node and restrict how much many edge-disjoint paths can go through it. Split each node into two as shown.

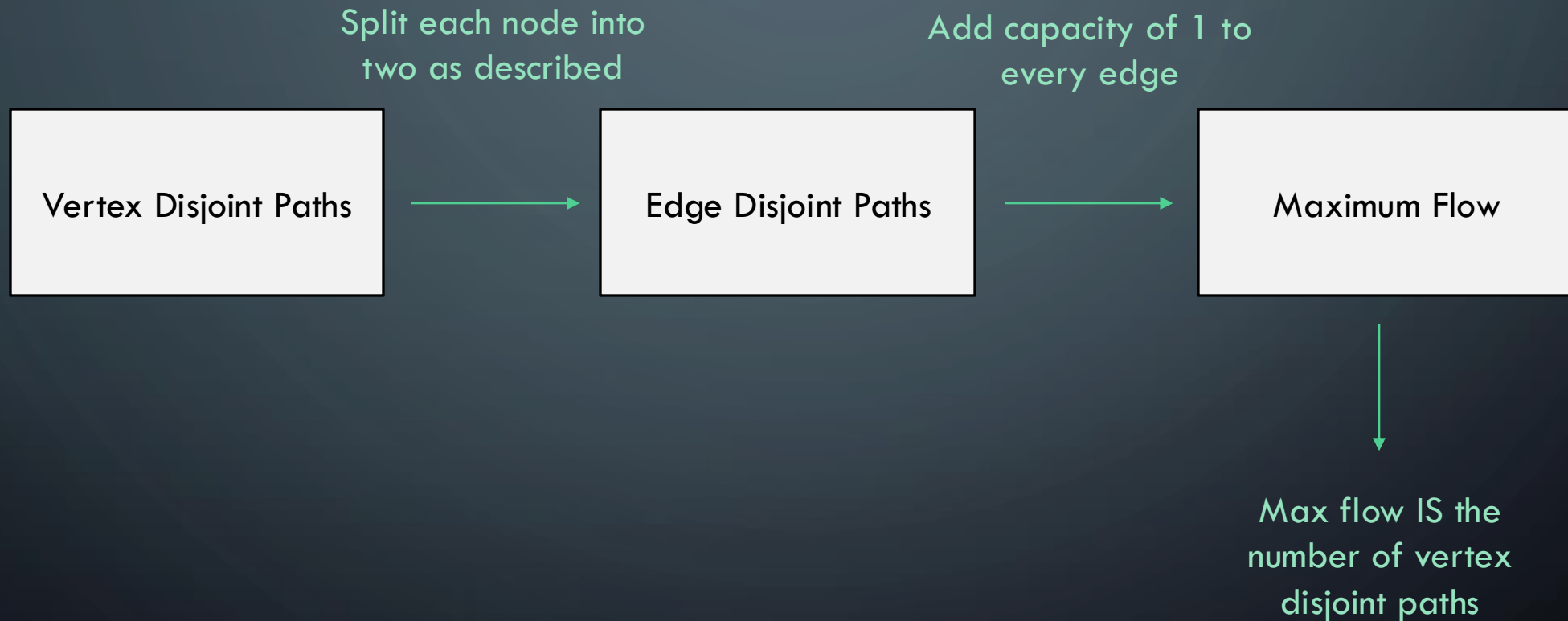


Why does this make sense. When finding edge disjoint paths, we can only use **one of the incoming edges**, **one of the outgoing edges**, and **the middle edge**.

WHAT'S THE SITUATION?



THE FINAL VERTEX DISJOINT PATH ALGORITHM



A decorative graphic on the left side of the slide, consisting of a network of thin, light-blue lines and small circles, resembling a circuit board or a neural network diagram. The lines are vertical and horizontal, with some diagonal connections, and the circles are placed at various points along these lines.

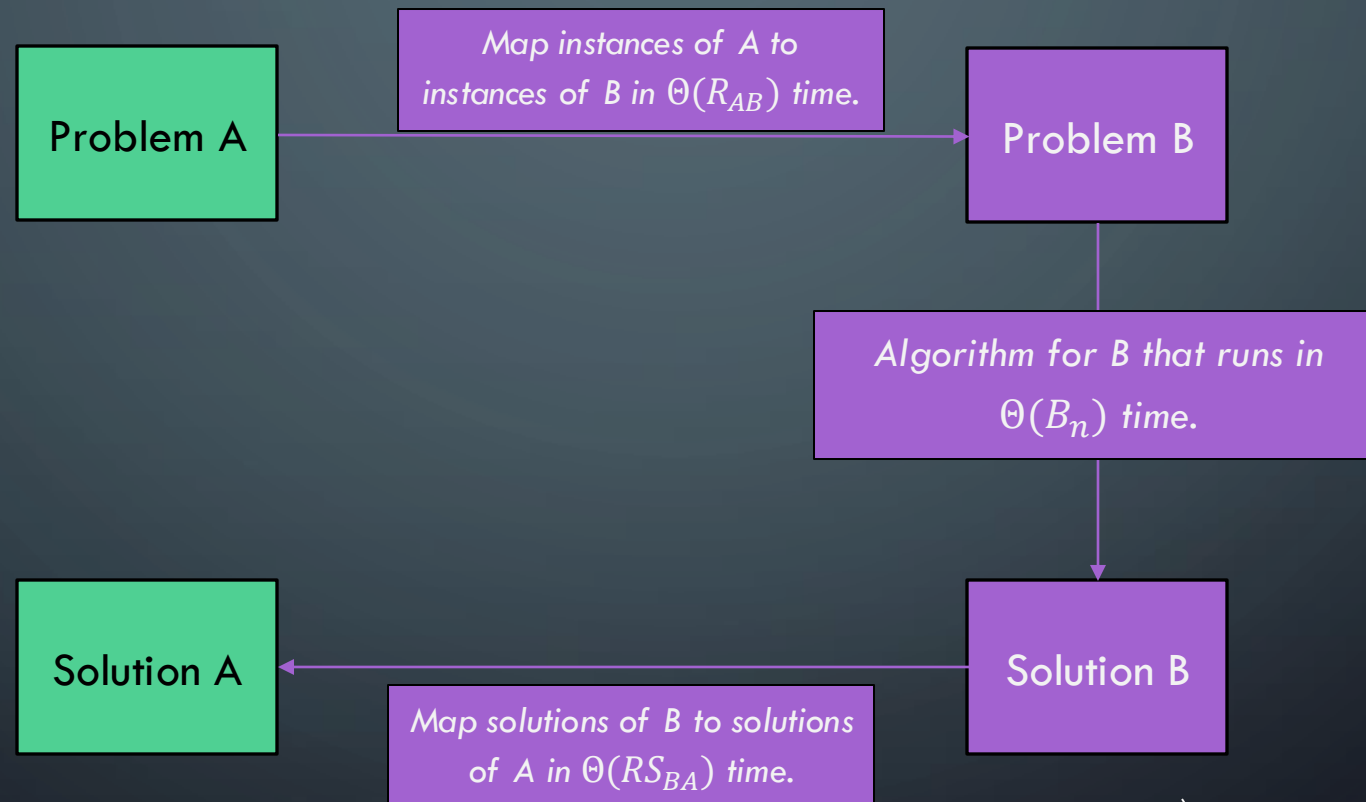
REDUCTIONS

ROADMAP: WHERE WE'RE GOING AND WHY

In Computer Science, we can often solve a new problem by casting it as an instance of another problem we already know how to solve. This is called a reduction.

MAPPING REDUCTION

A **reduction** uses a reduction function $R()$ to map instances of one problem (A) to instances of another problem (B) such that for any input string w , $A(w) == B(R(w))$



To solve A we use the purple path. Takes:
 $\Theta(R_{AB} + B_n + RS_{BA})$

A decorative graphic on the left side of the slide, consisting of a network of white lines and small circles on a dark blue background, resembling a circuit board or a neural network diagram.

BIPARTITE MATCHING

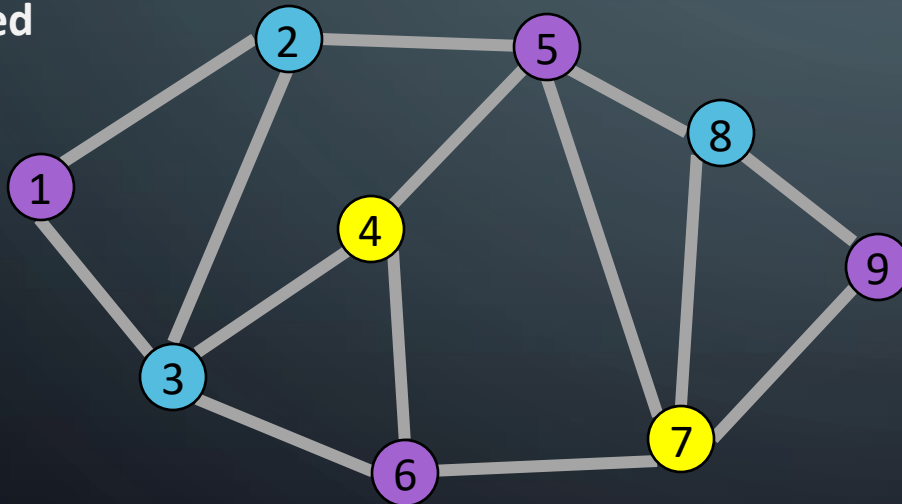
ANOTHER EXAMPLE OF A REDUCTION

REMINDER: BIPARTITE GRAPHS

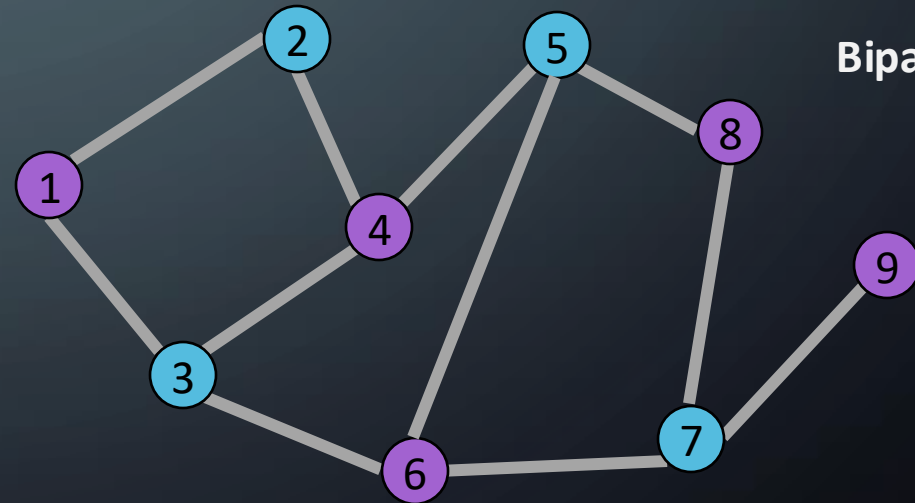
A (undirected) graph is **Bipartite** provided every vertex can be assigned to one of two teams such that every edge “crosses” teams

- Alternative: Every vertex can be given one of two colors such that no edges connect same-color nodes

Not Bipartite!
No color can
be assigned
to several
nodes!



Bipartite!



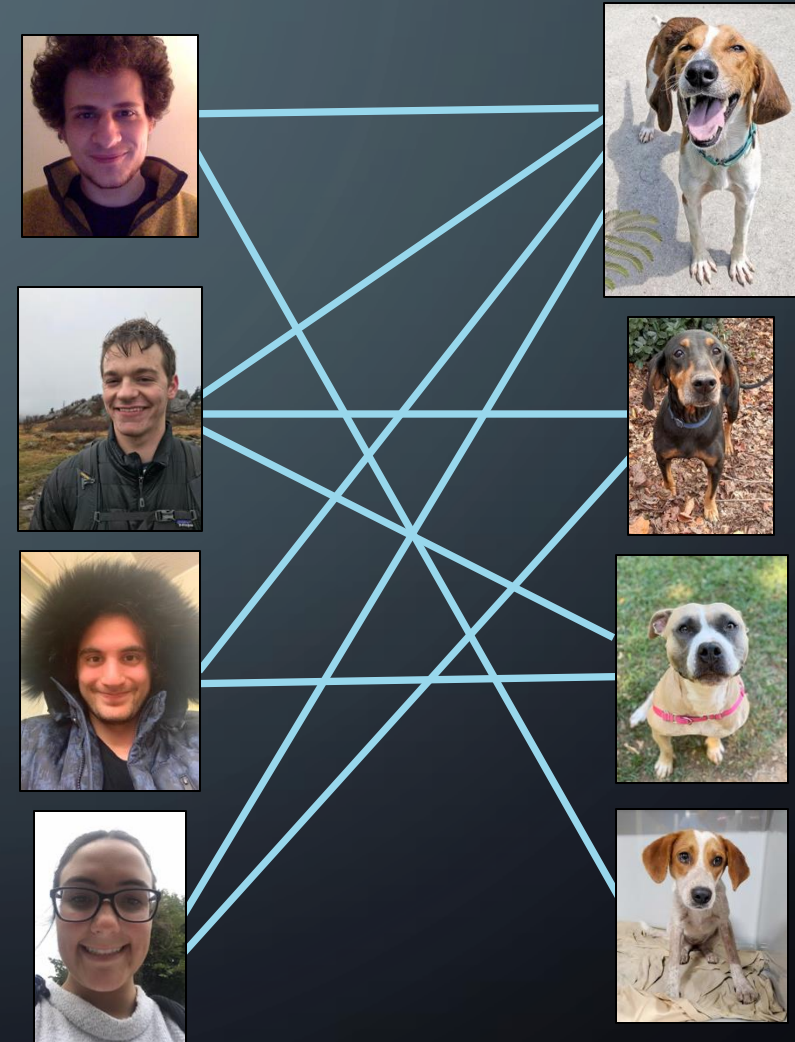
MAXIMUM BIPARTITE MATCHING

Maximum Bipartite Matching: Given a bipartite graph as input where the **edges represents candidate matches**, what is the **maximum number of "matches" that can be made in total?**

Example: Given a graph representing dog lovers and the dogs they are interested in adopting, what is the maximum number of adoptions that can be made (each dog lover can only adopt a dog they are interested in)

Dog Lovers

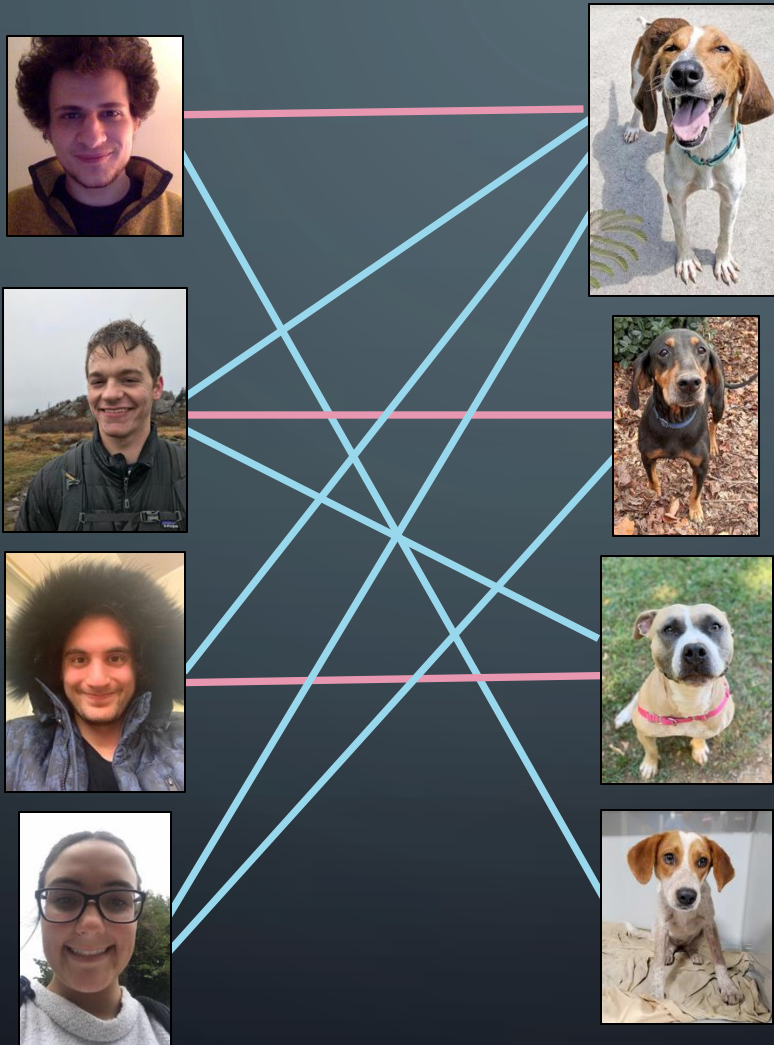
Adoptable Dogs



MAXIMUM BIPARTITE MATCHING

Dog Lovers

Adoptable Dogs

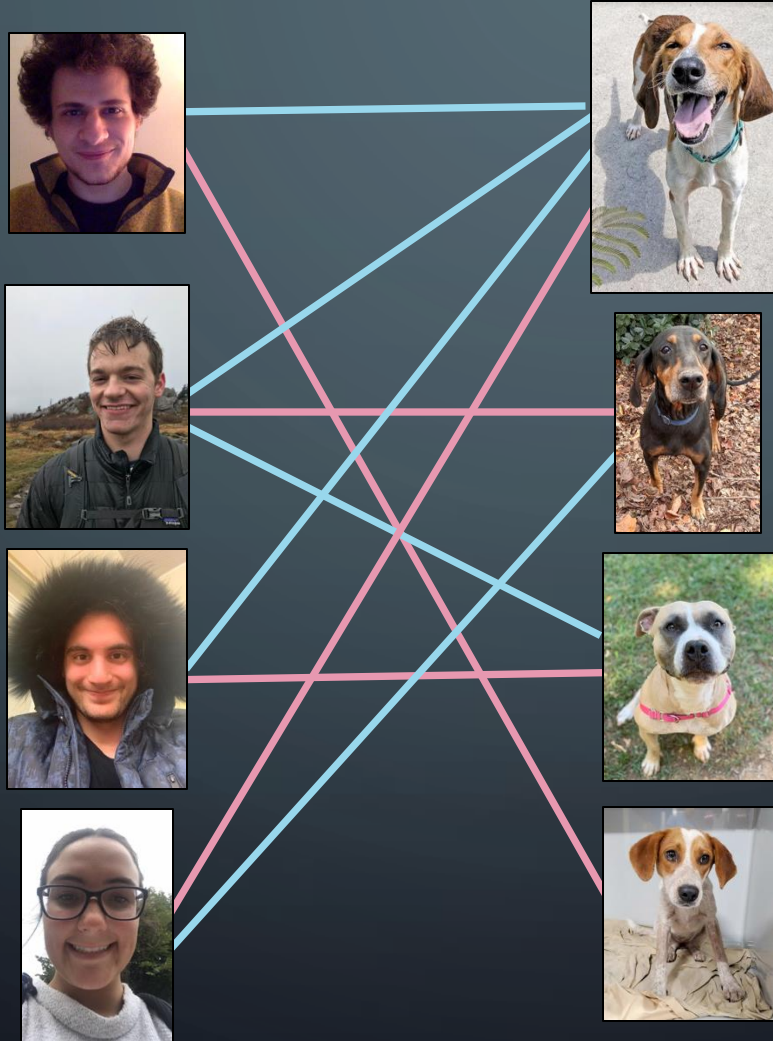


With this matching, three dogs get adopted. Is this maximum? Can we do four?

MAXIMUM BIPARTITE MATCHING

Dog Lovers

Adoptable Dogs



Better! In fact, the maximum possible!
How can we tell?

A **perfect bipartite match**:
Equal-sized left and right subsets, and all nodes have a matching edge

MAXIMUM BIPARTITE MATCHING

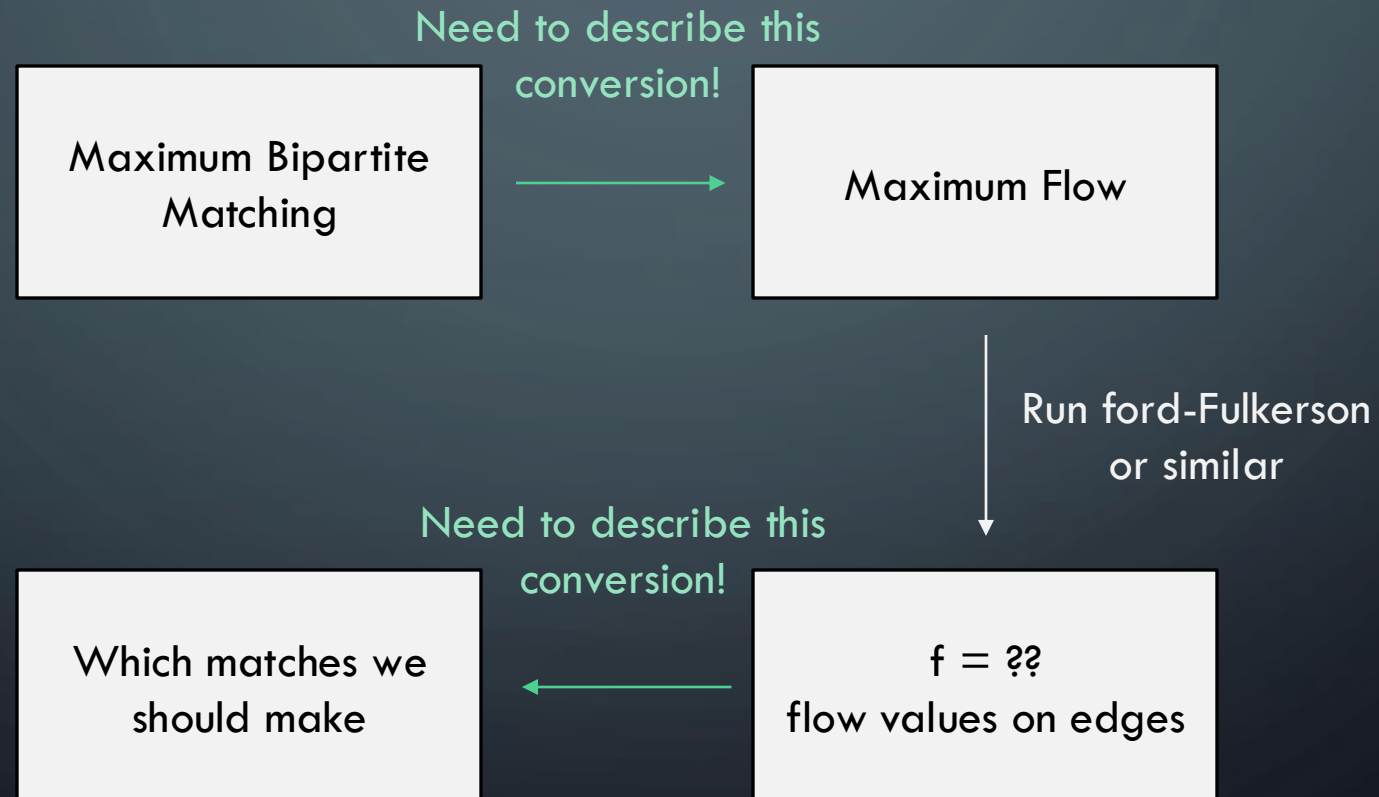
Given a graph $G = (L, R, E)$

a set of left nodes, right nodes, and edges between left and right

Find the largest set of edges $M \subseteq E$ such that each node $u \in L$ or $v \in R$ is incident to at most one edge.

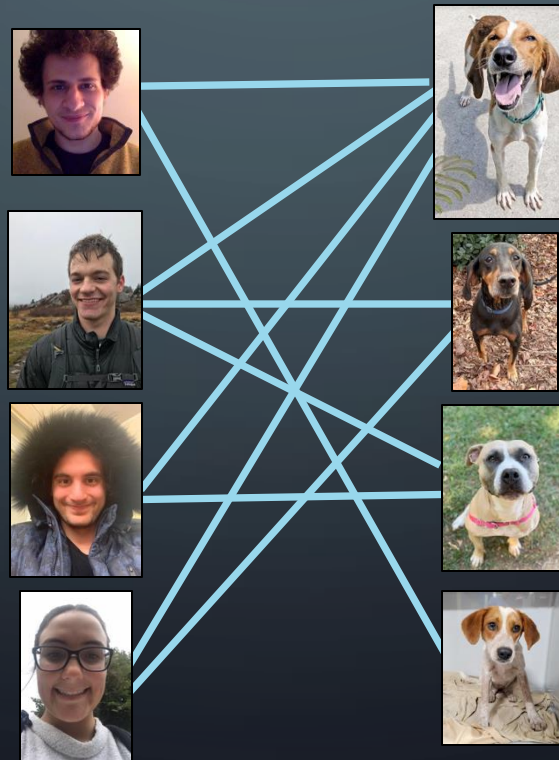
MAXIMUM BIPARTITE MATCHING USING MAX FLOW

We are going to show that maximum bipartite matching is reducible to max flow:



REDUCTION DETAILS

Make $G = (L, R, E)$ a flow network $G' = (V', E')$ by:

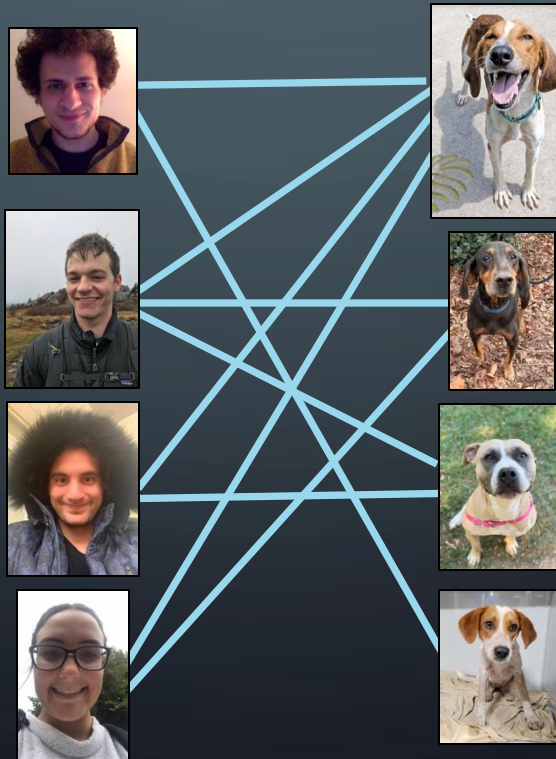


REDUCTION DETAILS

Make $G = (L, R, E)$ a flow network $G' = (V', E')$ by:

Add in a **source** and **sink** to the set of nodes:

$$V' = L \cup R \cup \{s, t\}$$



REDUCTION DETAILS

Make $G = (L, R, E)$ a flow network $G' = (V', E')$ by:

Add in a **source** and **sink** to the set of nodes:

$$V' = L \cup R \cup \{s, t\}$$

Add an **edge** from **source** to L and from R to **sink**:

$$E' = E \cup \{u \in L \mid (s, u)\} \cup \{v \in R \mid (v, t)\}$$



REDUCTION DETAILS

Make $G = (L, R, E)$ a flow network $G' = (V', E')$ by:

Add in a **source** and **sink** to the set of nodes:

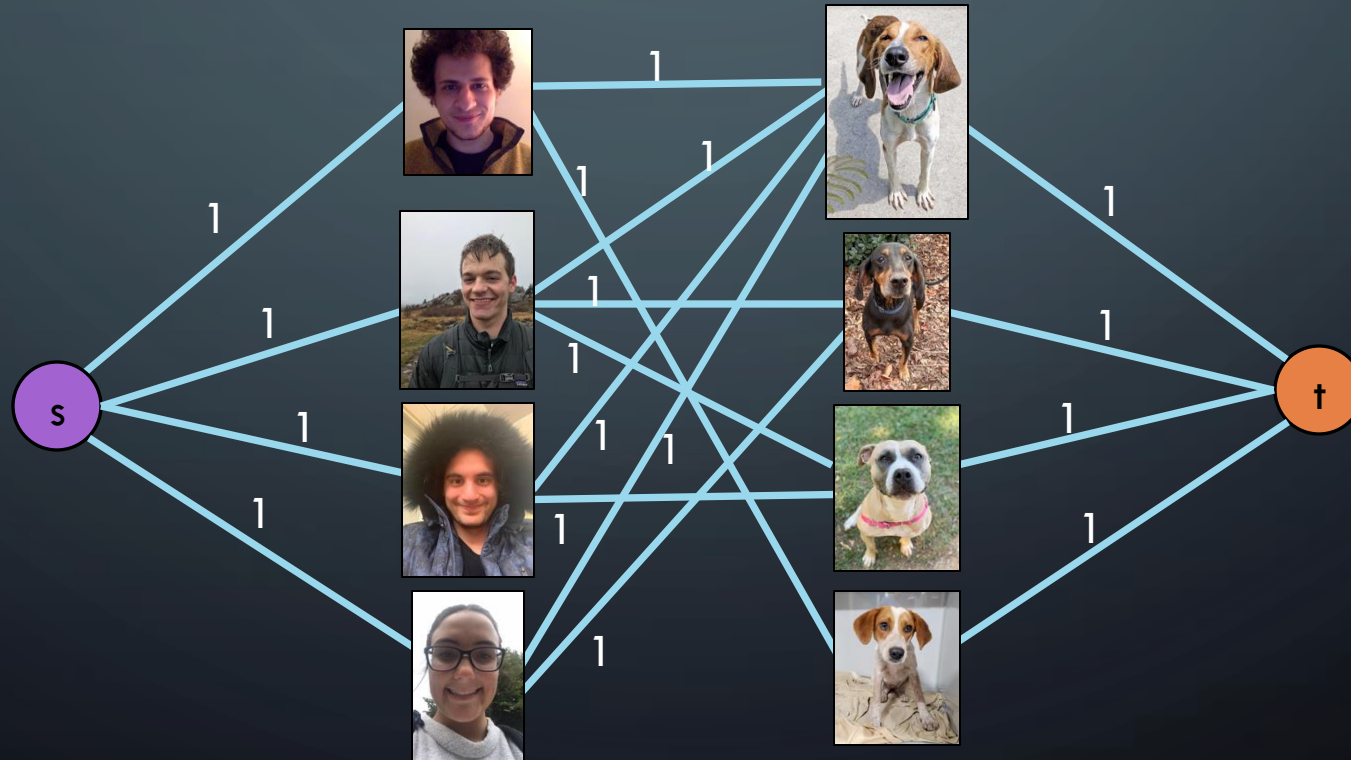
$$V' = L \cup R \cup \{s, t\}$$

Add an **edge** from **source** to L and from R to **sink**:

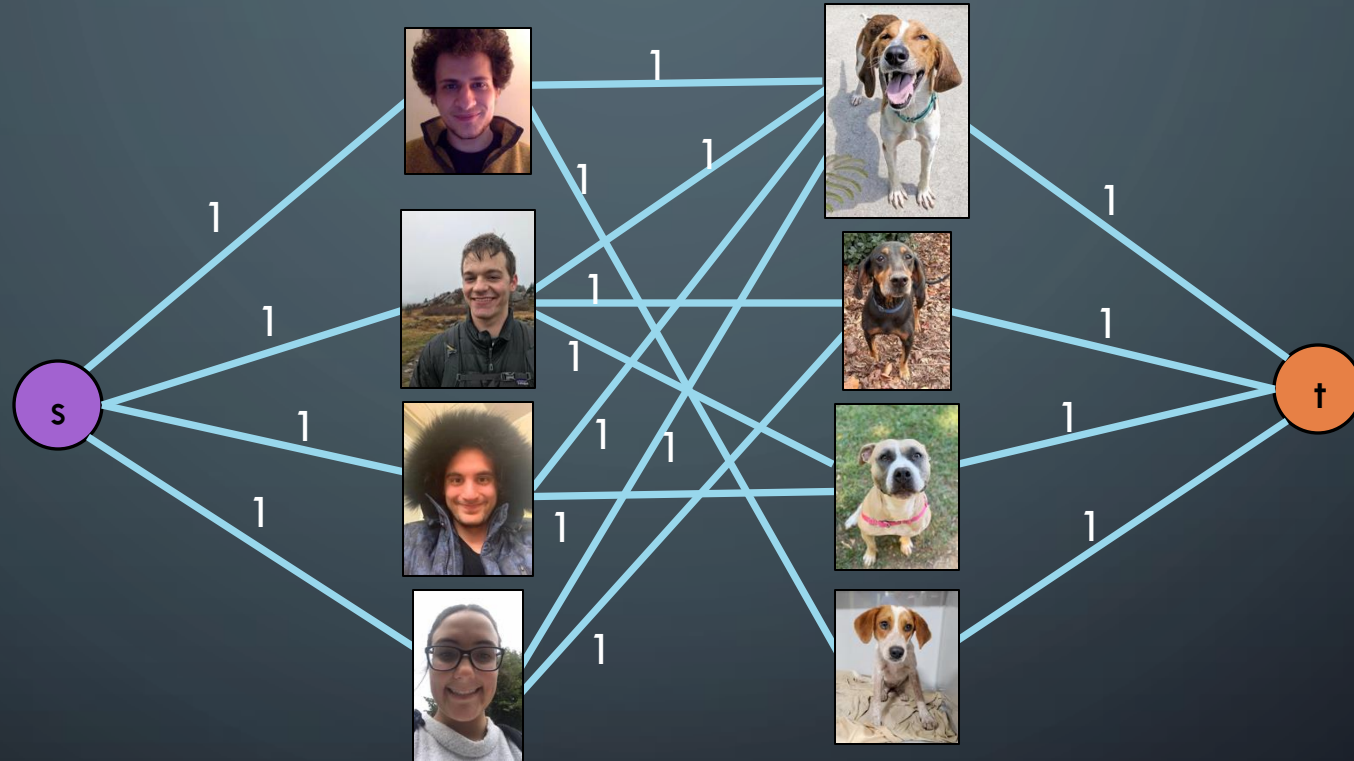
$$E' = E \cup \{u \in L \mid (s, u)\} \cup \{v \in R \mid (v, t)\}$$

Make each edge capacity 1:

$$\forall e \in E', c(e) = 1$$

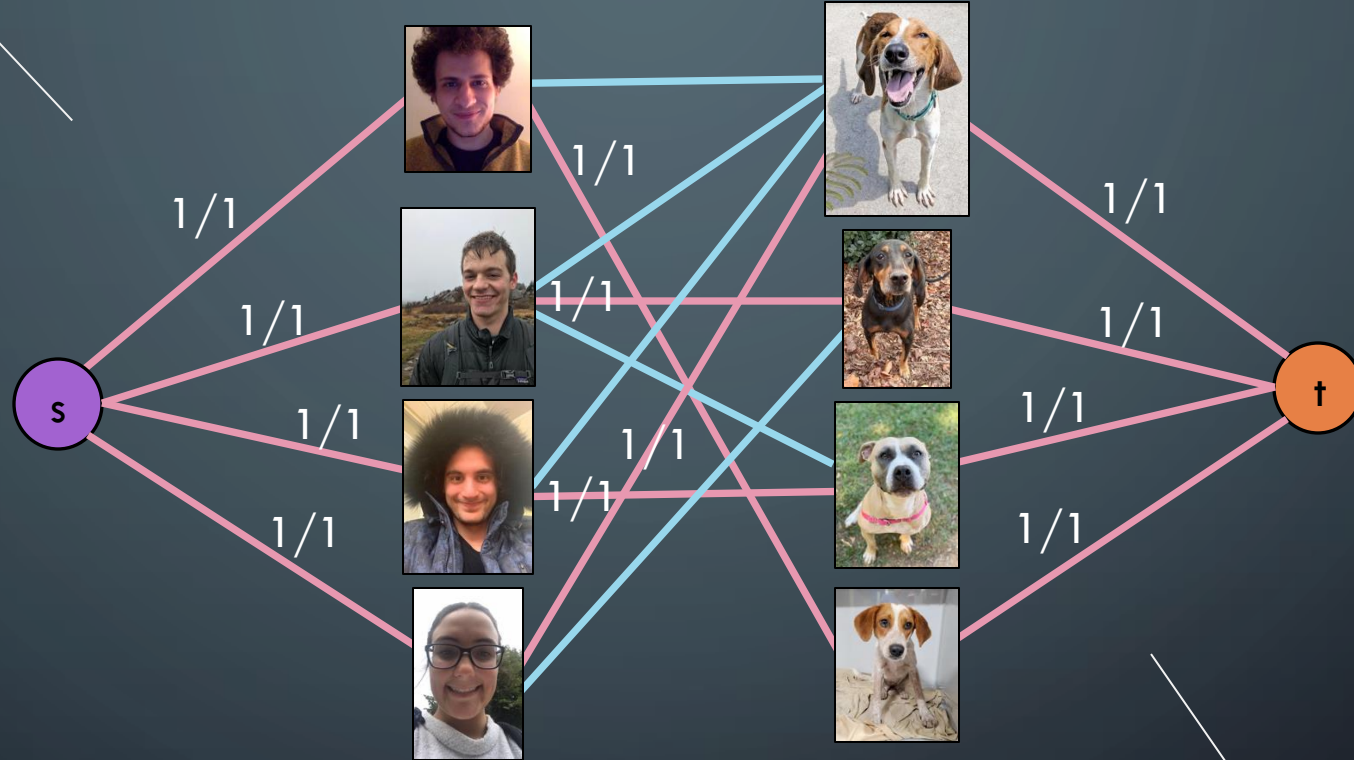


REDUCTION DETAILS



REDUCTION DETAILS

Call Ford-Fulkerson to push as much flow through this network as possible

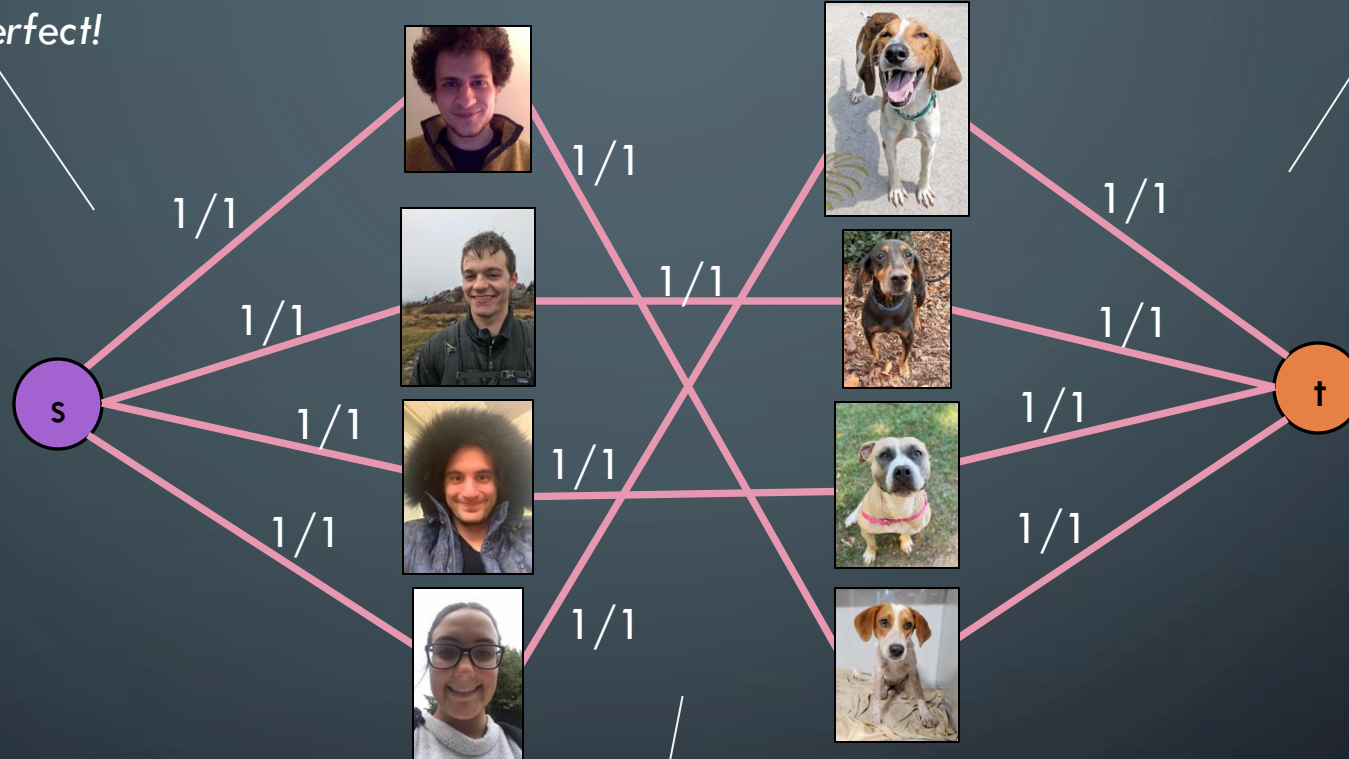


Red edges have flow running through, blue edges have 0 (number not shown to save space)

REDUCTION DETAILS

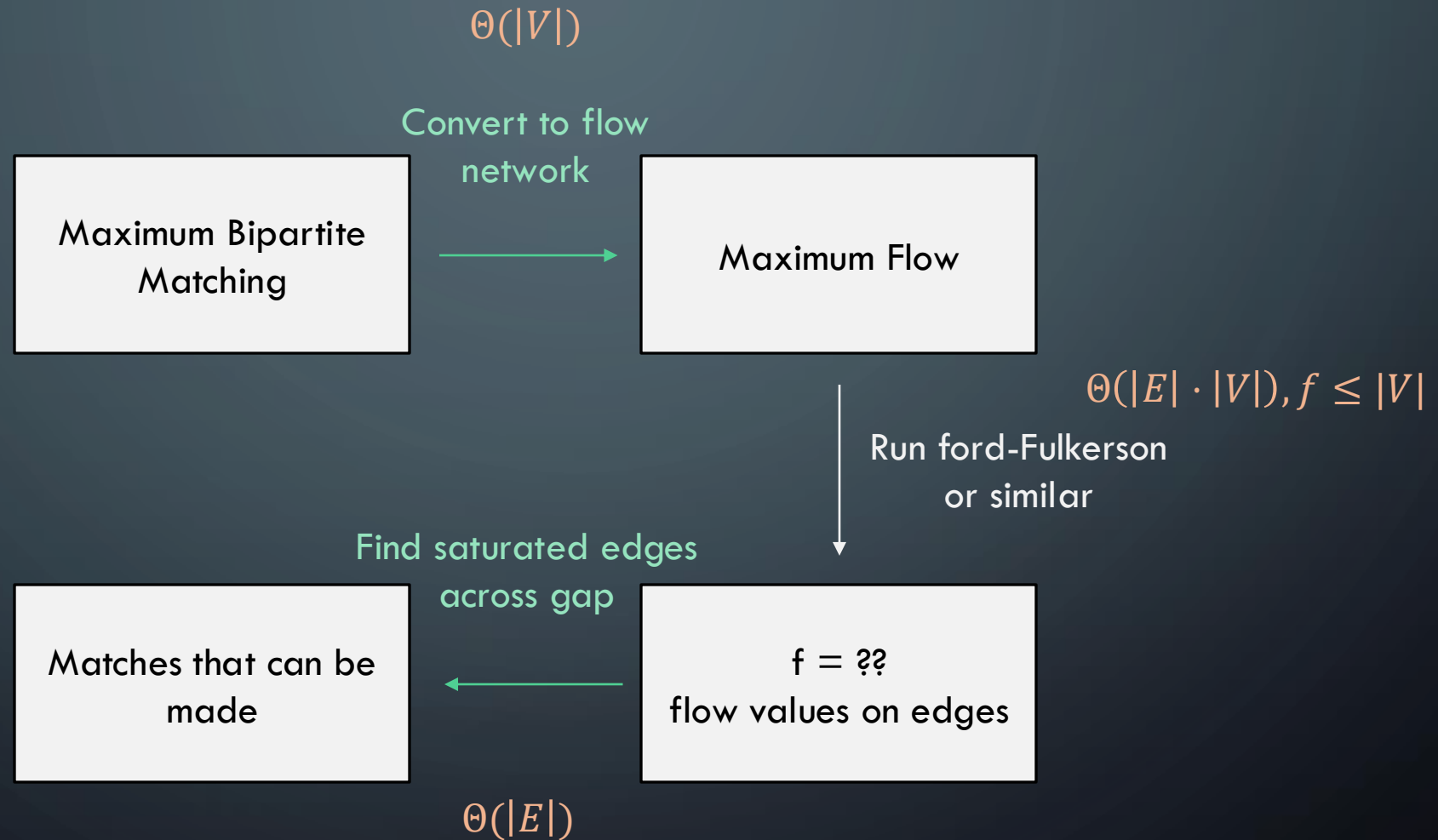
If max flow equals number of (original) nodes divided by 2, then the matching is perfect!

Note that in this example, all edges going into t (and out from s) are saturated, so this is a perfect matching



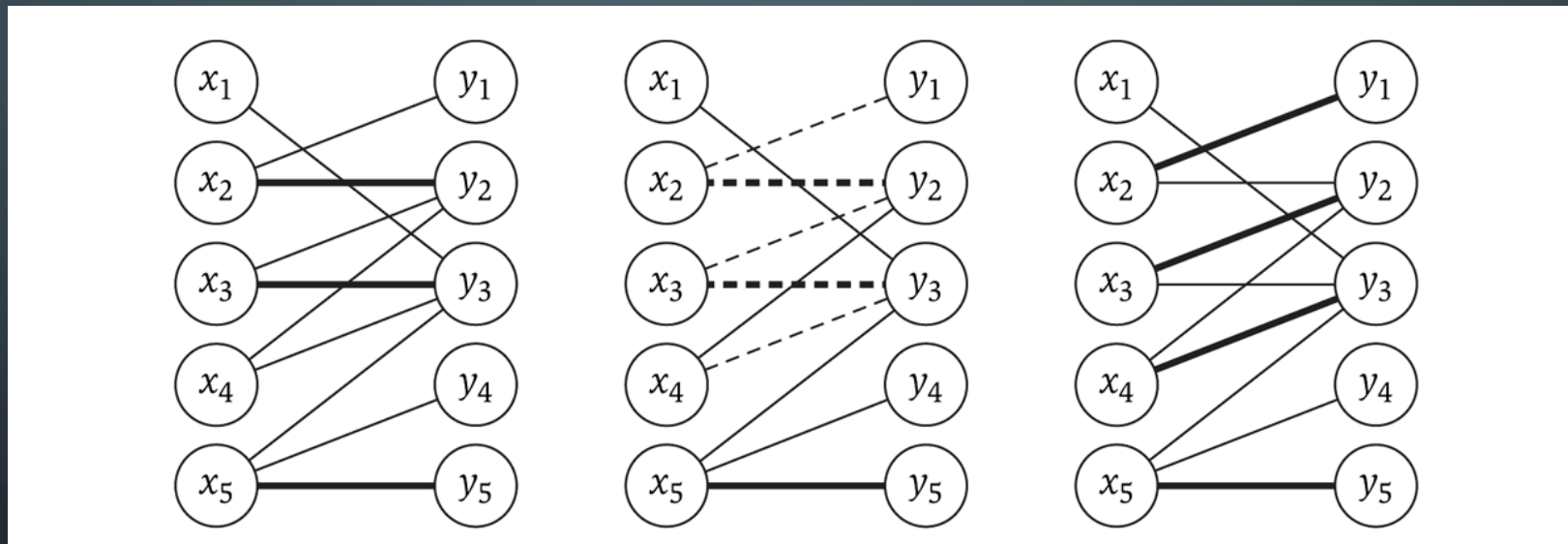
The edges going across gap that are saturated with flow represent the matches that we found!

MAXIMUM BIPARTITE MATCHING USING MAX FLOW



IMPERFECT BIPARTITE MATCHINGS

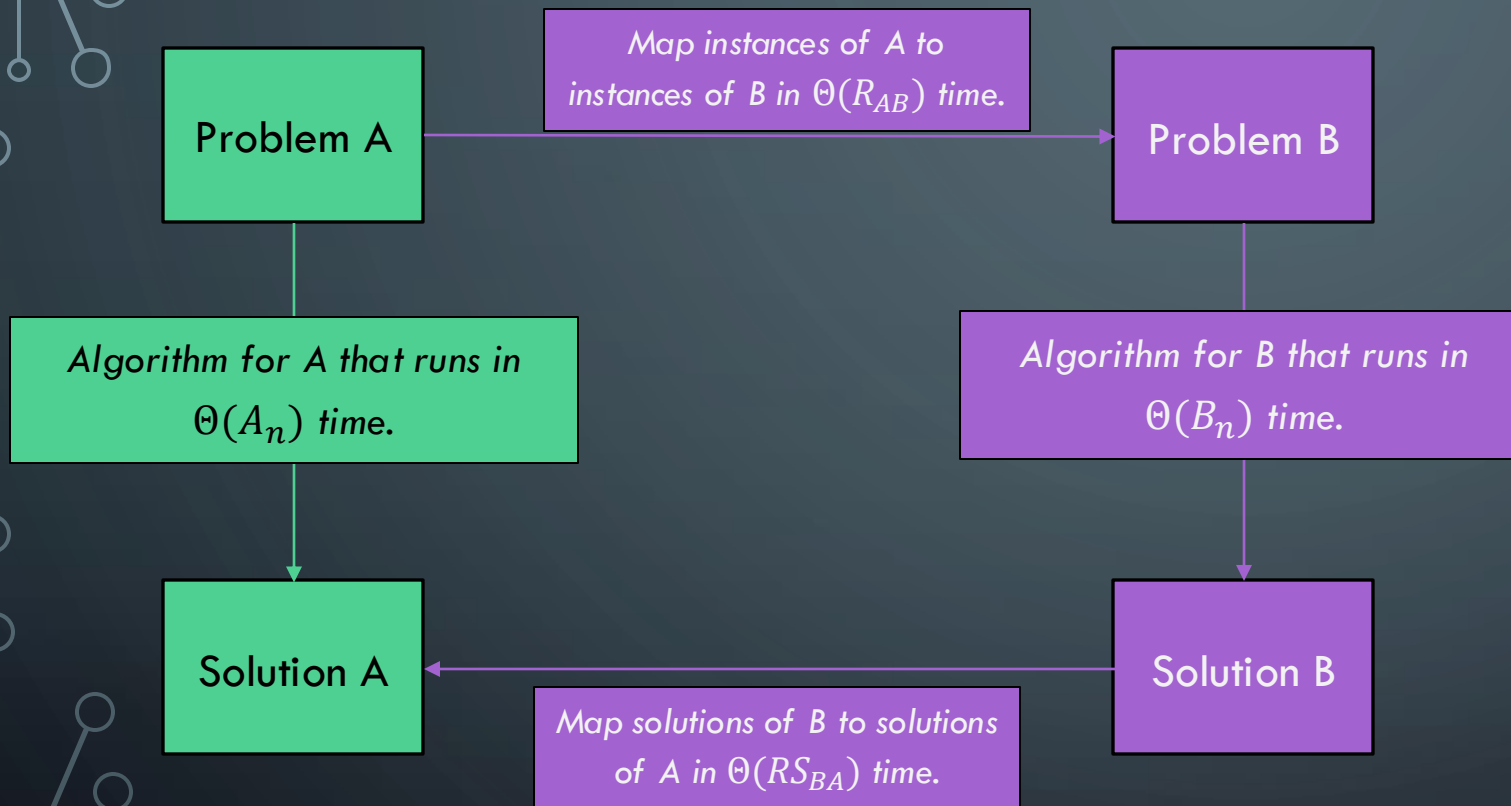
- These exist, and the algorithm may produce them, depending on the graph
 - The following shows an augmenting path (in the middle) used to achieve the maximal flow on the right





REDUCTIONS: ESTABLISHING RELATIVE SPEED / HARDNESS

RUNTIME COMPARISON



Which Algorithm is faster?

A_n

$R_{AB} + B_n + RS_{BA}$

If $R_{AB} + RS_{BA} \in O(B_n)$, then this represents a **valid reduction** and $A \leq_{R_{AB}+RS_{BA}} B$

If $R_{AB} + B_n + RS_{BA} \in O(A_n)$, then this is the best algorithm for A (or equally the best)

RUNTIME COMPARISON

Which Algorithm is faster?

A_n

$$R_{AB} + B_n + RS_{BA} \in \Theta(B_n)$$

B_n

A_n

Not surprisingly, if these two algorithms have same overall runtime, then either can be used (they are equivalent).

Harder Problems
(fastest algorithm
has slower runtime)

Easy Problems
(fastest algorithm has
very fast runtime)

RUNTIME COMPARISON

Which Algorithm is faster?

A_n

$$R_{AB} + B_n + RS_{BA} \in \Theta(B_n)$$

B_n

A_n

If solving A through reduction is SLOWER than directly solving A, this means problem B is simply harder than problem A (but the reduction is still valid)

Harder Problems
(fastest algorithm has slower runtime)

Easy Problems
(fastest algorithm has very fast runtime)

RUNTIME COMPARISON

Which Algorithm is faster?

A_n

$$R_{AB} + B_n + RS_{BA} \in \Theta(B_n)$$

A_n

B_n

If the reduction is FASTER than directly solving A, What does this mean? It means the reduction IS the best way to solve A (and this picture doesn't make sense)

Harder Problems
(fastest algorithm
has slower runtime)

Easy Problems
(fastest algorithm has
very fast runtime)

RUNTIME COMPARISON

Which Algorithm is faster?

$$A_n$$

$$R_{AB} + B_n + RS_{BA} \in \Theta(B_n)$$

OLD A_n

$$A_n = B_n$$

...and the direct algorithm for A is
obsolete. The reduction through problem
B is the direct way to solve A

Harder Problems
(fastest algorithm
has slower runtime)

Easy Problems
(fastest algorithm has
very fast runtime)

RUNTIME COMPARISON

Suppose time goes on, and somebody find a FASTER way to solve B in B'_n time, how will the picture to the right change as a result?

A now has a faster algorithm also! So improving B 's algorithm improves A 's. They are linked in this direction!

$$A_n = B_n$$

$$A'_n = R_{AB} + B'_n + RS_{BA}$$

This is ONLY true if the reduction stays valid, meaning the conversion is still fast: $R_{AB} + RS_{BA} \in O(B'_n)$

Harder Problems
(fastest algorithm has slower runtime)

Easy Problems
(fastest algorithm has very fast runtime)

RUNTIME COMPARISON

Now suppose time goes on and someone finds a VERY fast algorithm for A. What could happen?

Now, the reduction may still be valid, but we are back to B being strictly harder than A

B'_n

A'_n

*Harder Problems
(fastest algorithm
has slower runtime)*

*Easy Problems
(fastest algorithm has
very fast runtime)*

A decorative graphic on the left side of the slide, consisting of a network of thin, light-blue lines and small circles, resembling a circuit board or a neural network diagram. The lines are vertical and horizontal, with some diagonal connections, and the circles are small and white, acting as nodes or junctions.

USING REDUCTIONS FOR LOWER- BOUNDS PROOF

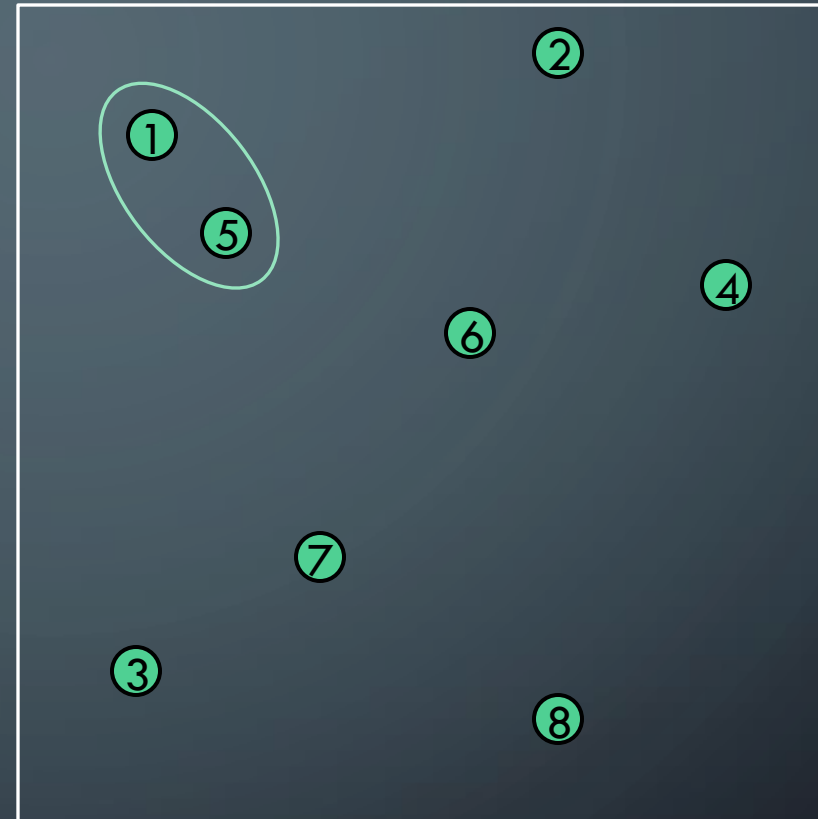
CLOSEST PAIR OF POINTS IN 2D SPACE

Given:

A list of points in 2D space

Return:

Distance of the pair of points that are closest together (and the points themselves too)

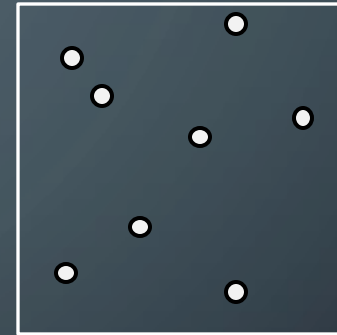


We solved this in $\Theta(n \log n)$ during our divide-and-conquer unit! Is that the best we can do?

USING REDUCTIONS TO PROVE LOWER-BOUNDS

Claim: Closest pair of points is $\Omega(n \log n)$

Proof Approach:



Assume f.s.o.c. solvable
in $o(n \log n)$

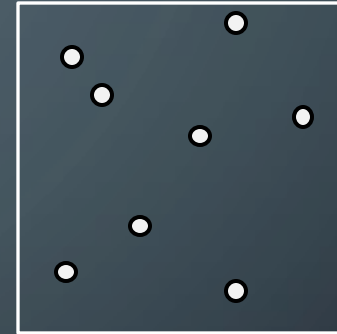


USING REDUCTIONS TO PROVE LOWER-BOUNDS

Claim: Closest pair of points is $\Omega(n \log n)$

Proof Approach:

Find a problem
that we already
know is $\Omega(n \log n)$



Assume f.s.o.c. solvable
in $o(n \log n)$



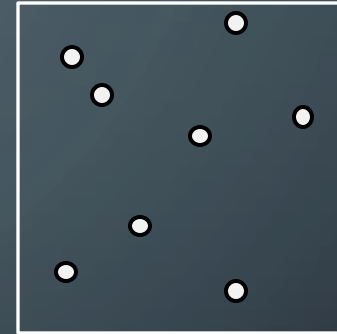
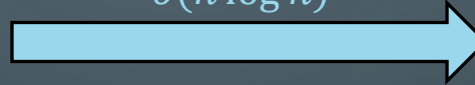
USING REDUCTIONS TO PROVE LOWER-BOUNDS

Claim: Closest pair of points is $\Omega(n \log n)$

Proof Approach:

Find a problem
that we already
know is $\Omega(n \log n)$

Describe fast
conversion to CPP in
 $o(n \log n)$



Assume f.s.o.c. solvable
in $o(n \log n)$



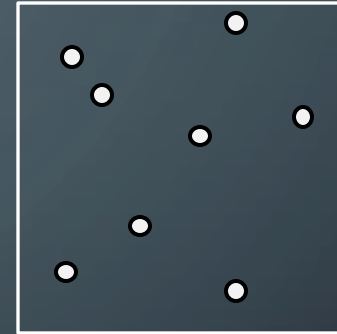
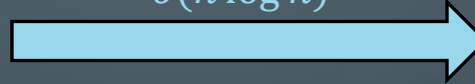
USING REDUCTIONS TO PROVE LOWER-BOUNDS

Claim: Closest pair of points is $\Omega(n \log n)$

Proof Approach:

Find a problem
that we already
know is $\Omega(n \log n)$

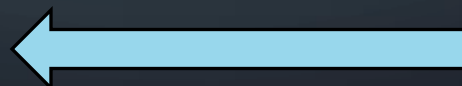
Describe fast
conversion to CPP in
 $o(n \log n)$



Assume f.s.o.c. solvable
in $o(n \log n)$



Convert answer fast,
in $o(n \log n)$



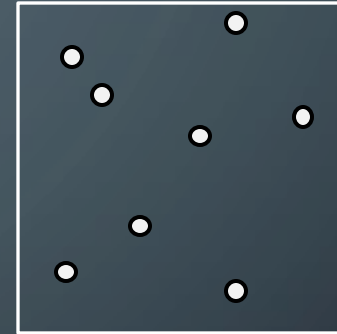
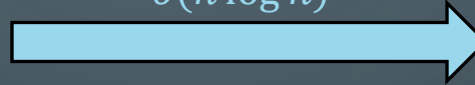
USING REDUCTIONS TO PROVE LOWER-BOUNDS

Claim: Closest pair of points is $\Omega(n \log n)$

Proof Approach:

Find a problem
that we already
know is $\Omega(n \log n)$

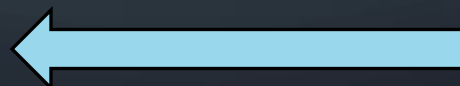
Describe fast
conversion to CPP in
 $o(n \log n)$



Assume f.s.o.c. solvable
in $o(n \log n)$



Convert answer fast,
in $o(n \log n)$



Solution found in
 $o(n \log n)$



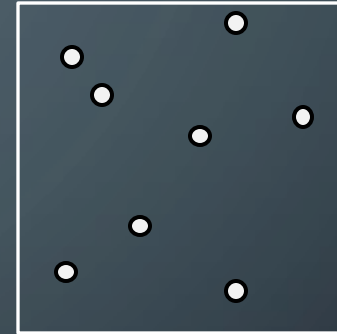
USING REDUCTIONS TO PROVE LOWER-BOUNDS

Claim: Closest pair of points is $\Omega(n \log n)$

Proof Approach:

Find a problem
that we already
know is $\Omega(n \log n)$

Describe fast
conversion to CPP in
 $o(n \log n)$



Assume f.s.o.c. solvable
in $o(n \log n)$



Convert answer fast,
in $o(n \log n)$

Solution found in
 $o(n \log n)$

Contradiction! Each step of algorithm
is $o(n \log n)$ so solution found faster
than is theoretically possible!

USING REDUCTIONS TO PROVE LOWER-BOUNDS

Find a problem
that we already
know is $\Omega(n \log n)$

Element Uniqueness Problem: Given a list of numbers, are all of the elements unique?

25	88	13	90	64	14	18	99	71	77	39
----	----	----	----	----	----	----	----	----	----	----

TRUE

25	88	13	90	64	14	88	99	71	77	39
----	----	----	----	----	----	----	----	----	----	----

FALSE

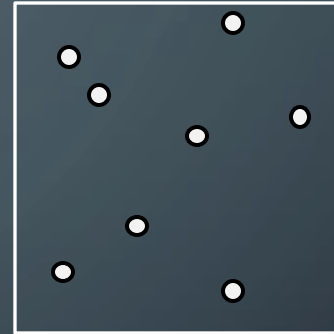
Theorem: The element distinctness problem is $\Omega(n \log n)$
(just trust us for now...not enough time to get into it)

USING REDUCTIONS TO PROVE LOWER-BOUNDS

Claim: Closest pair of points is $\Omega(n \log n)$

Proof Approach:

Element Distinctness
 $= \Omega(n \log n)$



USING REDUCTIONS TO PROVE LOWER-BOUNDS

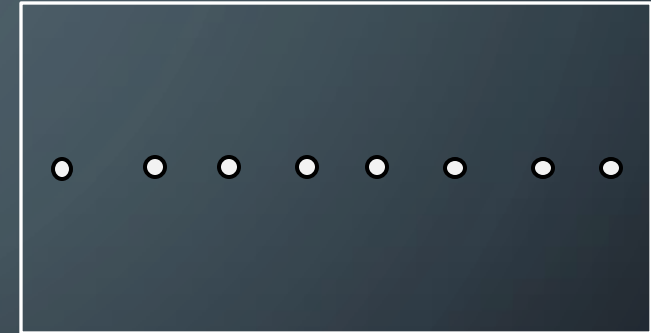
Claim: Closest pair of points is $\Omega(n \log n)$

Proof Approach:

Element Distinctness
 $= \Omega(n \log n)$

For each element x ,
create point $(x, 0)$

$\Theta(n)$



USING REDUCTIONS TO PROVE LOWER-BOUNDS

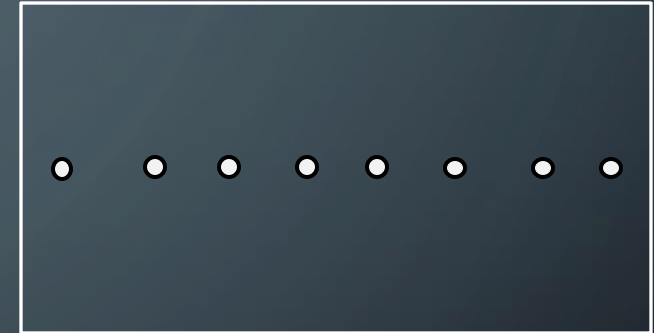
Claim: Closest pair of points is $\Omega(n \log n)$

Proof Approach:

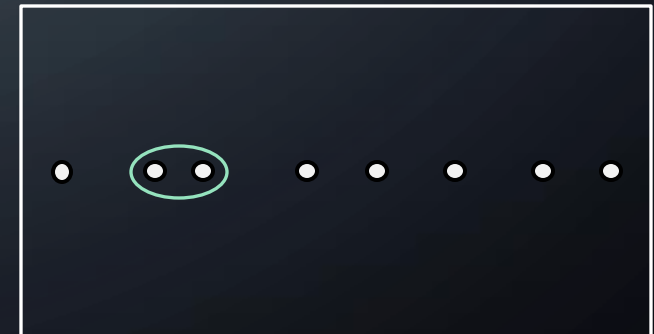
Element Distinctness
 $= \Omega(n \log n)$

For each element x ,
create point $(x, 0)$

$\Theta(n)$



Assume f.s.o.c. solvable
in $o(n \log n)$



USING REDUCTIONS TO PROVE LOWER-BOUNDS

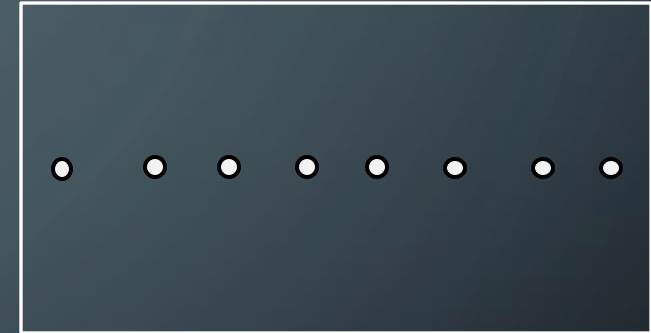
Claim: Closest pair of points is $\Omega(n \log n)$

Proof Approach:

Element Distinctness
 $= \Omega(n \log n)$

For each element x ,
create point $(x, 0)$

$\Theta(n)$



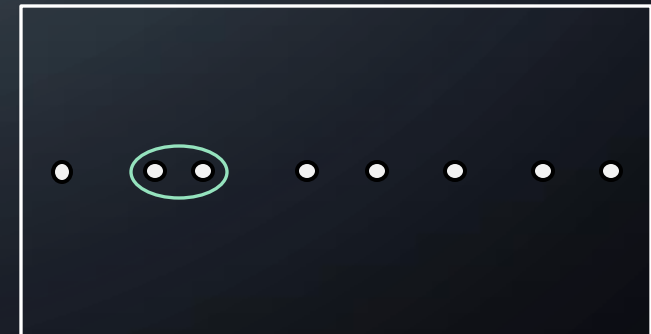
Assume f.s.o.c. solvable
in $o(n \log n)$



```
If cpp distance == 0  
    return false;  
else  
    return true
```

Solution found in
 $o(n \log n)$

$\Theta(1)$



USING REDUCTIONS TO PROVE LOWER-BOUNDS

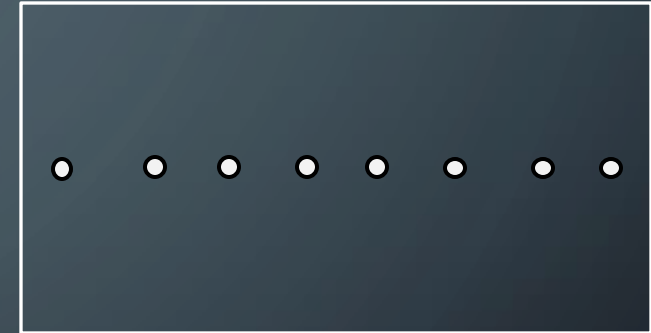
Claim: Closest pair of points is $\Omega(n \log n)$

Proof Approach:

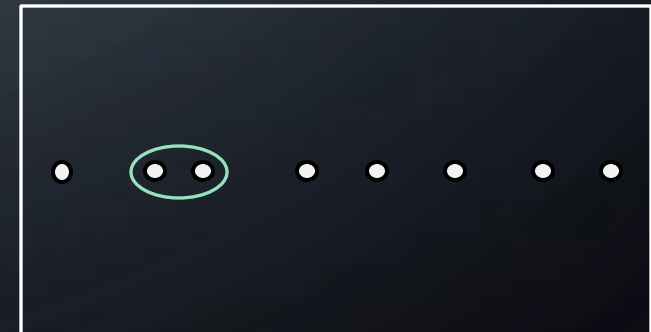
Element Distinctness
 $= \Omega(n \log n)$

For each element x ,
create point $(x, 0)$

$\Theta(n)$



Assume f.s.o.c. solvable
in $o(n \log n)$



```
If cpp distance == 0  
    return false;  
else  
    return true
```

$\Theta(1)$

Solution found in
 $o(n \log n)$

Contradiction! Each step of algorithm
is $o(n \log n)$ so solution found faster
than is theoretically possible!

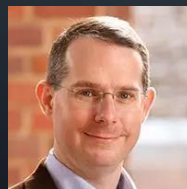
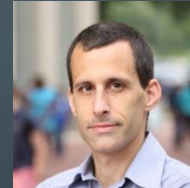


ANOTHER EXAMPLE OF
REDUCTION: IND. SET AND VERT.
COVER.

A decorative graphic on the left side of the slide, consisting of a network of white lines and small circles on a dark blue background, resembling a circuit board or a neural network diagram.

PROBLEM A: INDEPENDENT SET

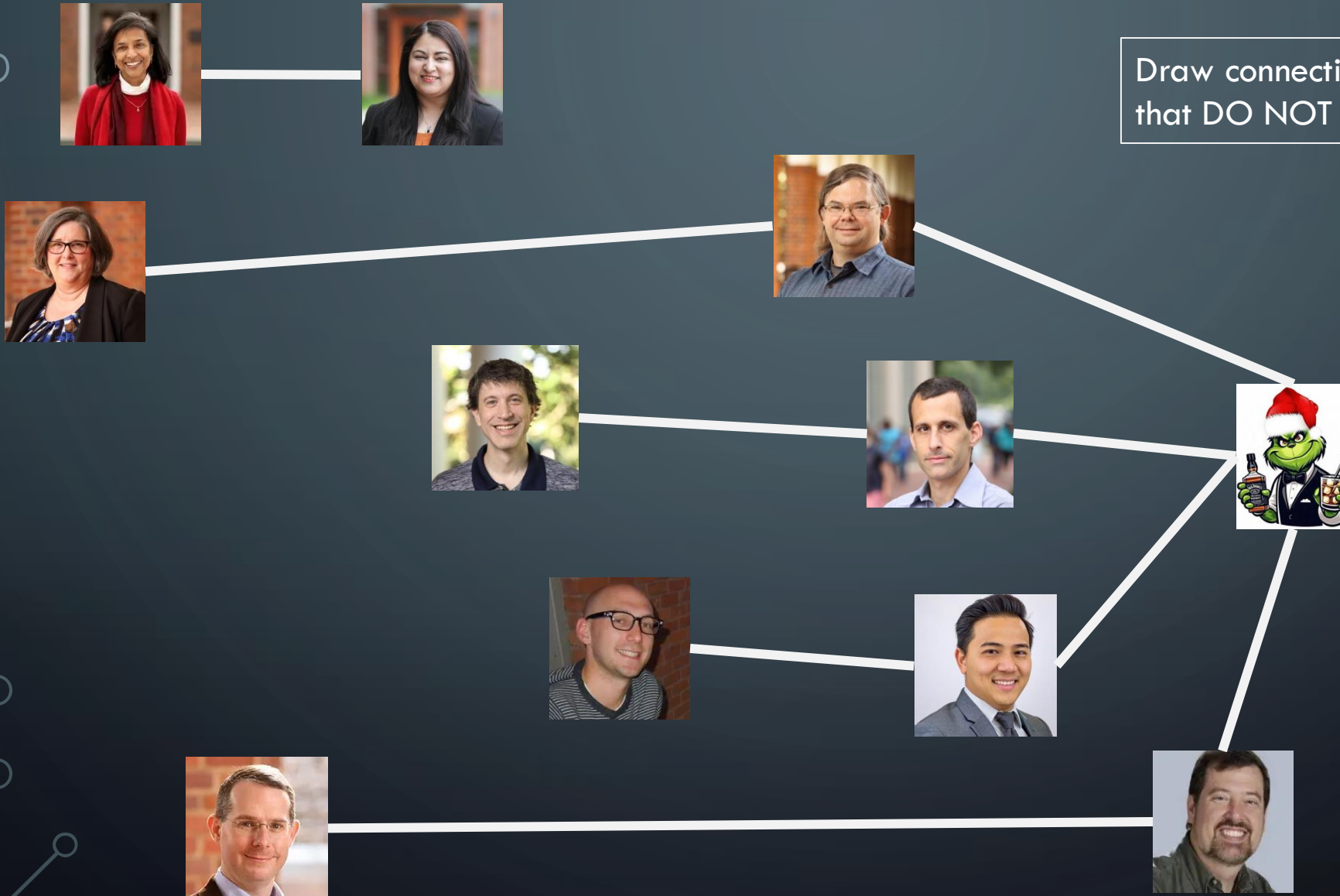
PROBLEM A: PARTY PROBLEM



A bunch of CS faculty members are going to a party! But...some faculty members DO NOT get along with one another.

PROBLEM A: PARTY PROBLEM

Draw connections between members that DO NOT get along

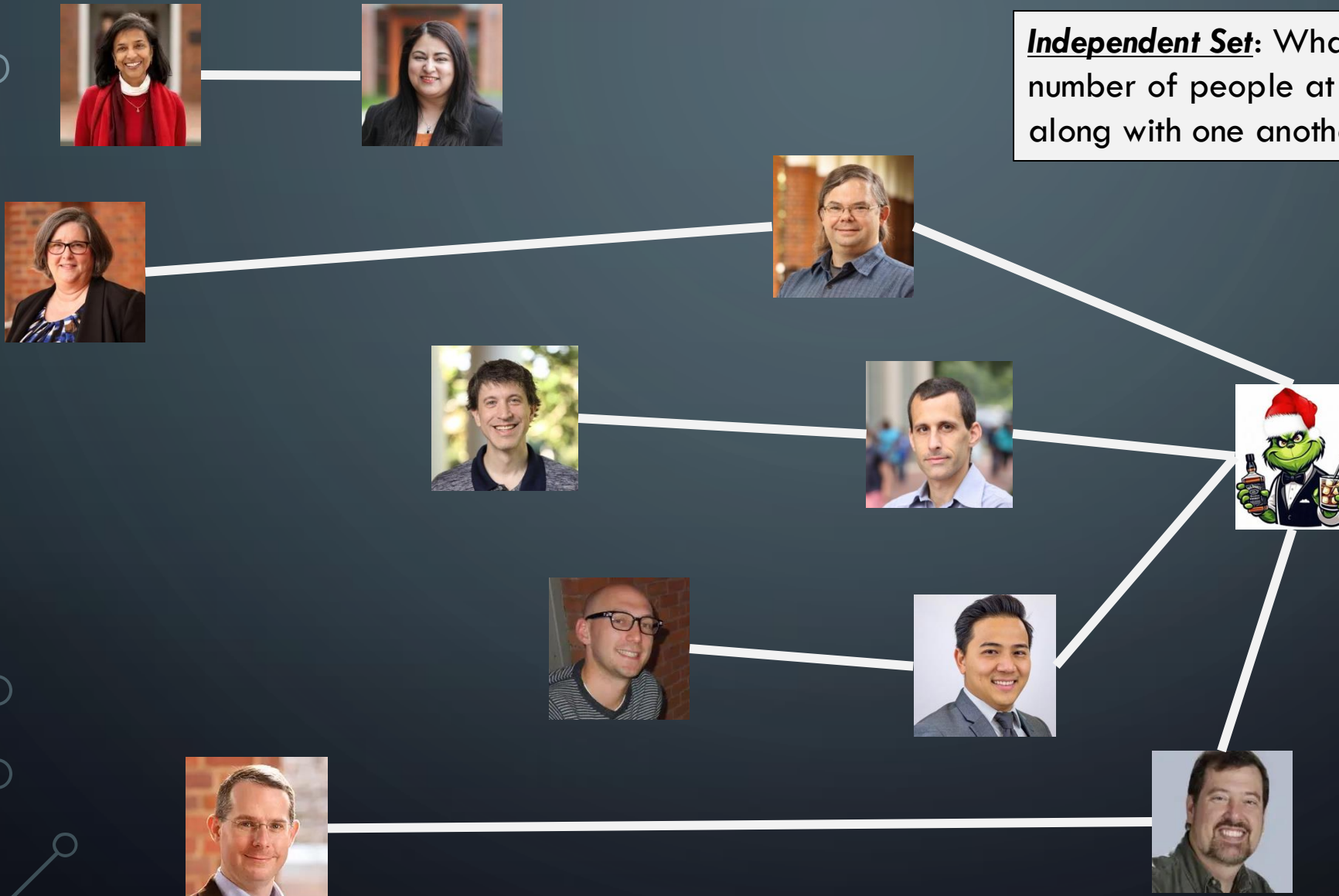


PROBLEM A: PARTY PROBLEM

Independent Set: What is the MAXIMUM number of people at this party who ALL get along with one another

```
graph LR; A[Red Jacket] --- B[Long Hair]; C[Grey Hair] --- D[Blue Shirt]; D --- E[Grinch]; F[Brown Hair] --- E; G[Dark Hair] --- E; H[Striped Shirt] --- I[Suit]; I --- E; E --- J[Beard]; A --- C; B --- F
```

Independent Set: What is the MAXIMUM number of people at this party who ALL get along with one another



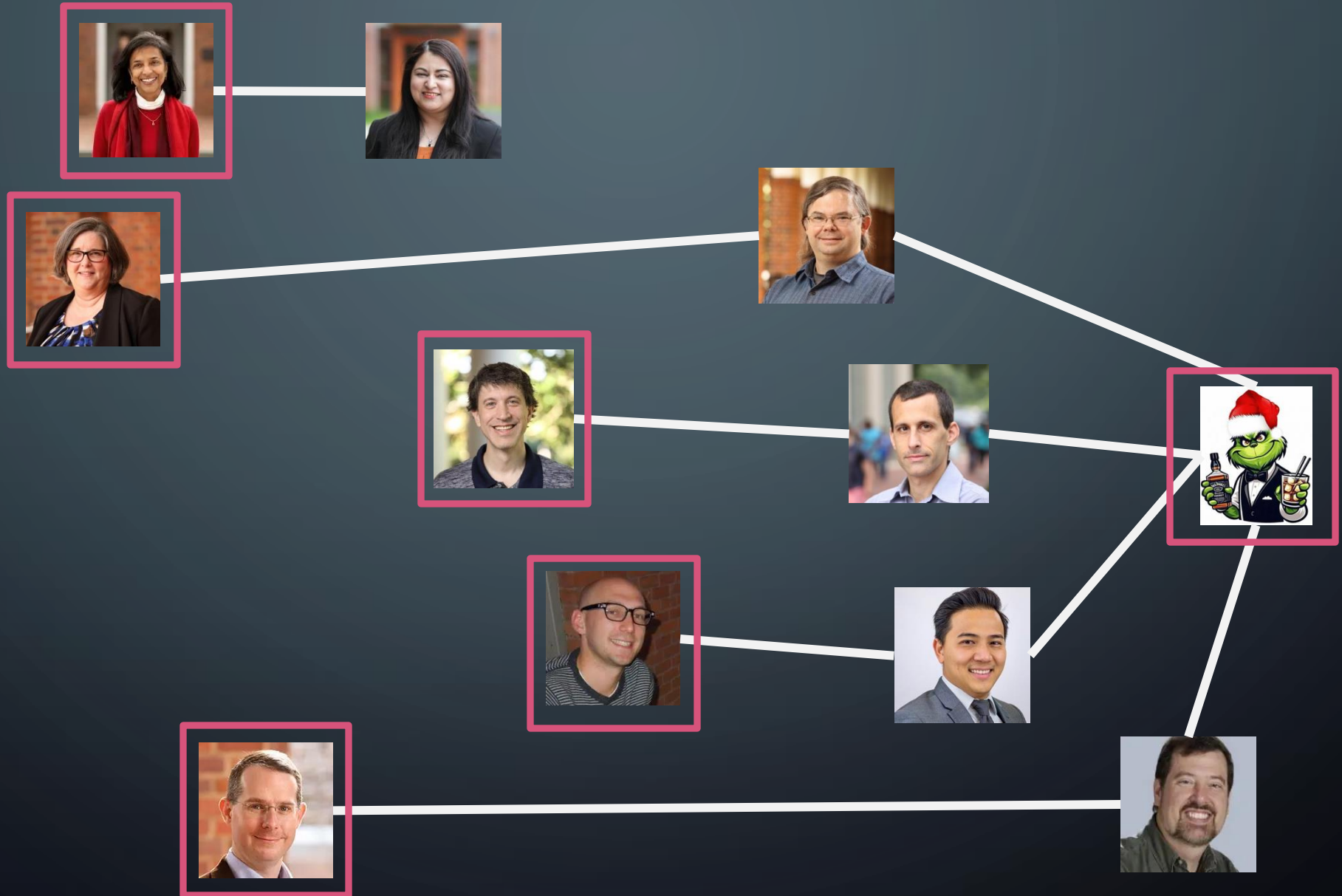
PROBLEM A: MAX INDEPENDENT SET

Maximum Independent Set: Given a graph $G = (V, E)$, what is the maximum set of nodes in a subset $S \subseteq V$ such that no edge connects two nodes in S

PROBLEM A: MAX INDEPENDENT SET



PROBLEM A: MAX INDEPENDENT SET



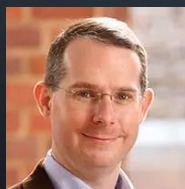
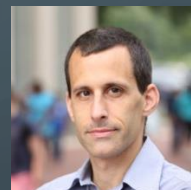
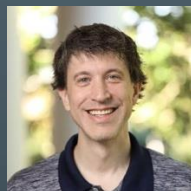
A decorative graphic on the left side of the slide, consisting of a network of white lines and small circles on a dark blue background, resembling a circuit board or a neural network structure.

PROBLEM B: VERTEX COVER

PROBLEM B: MEETING NOTES

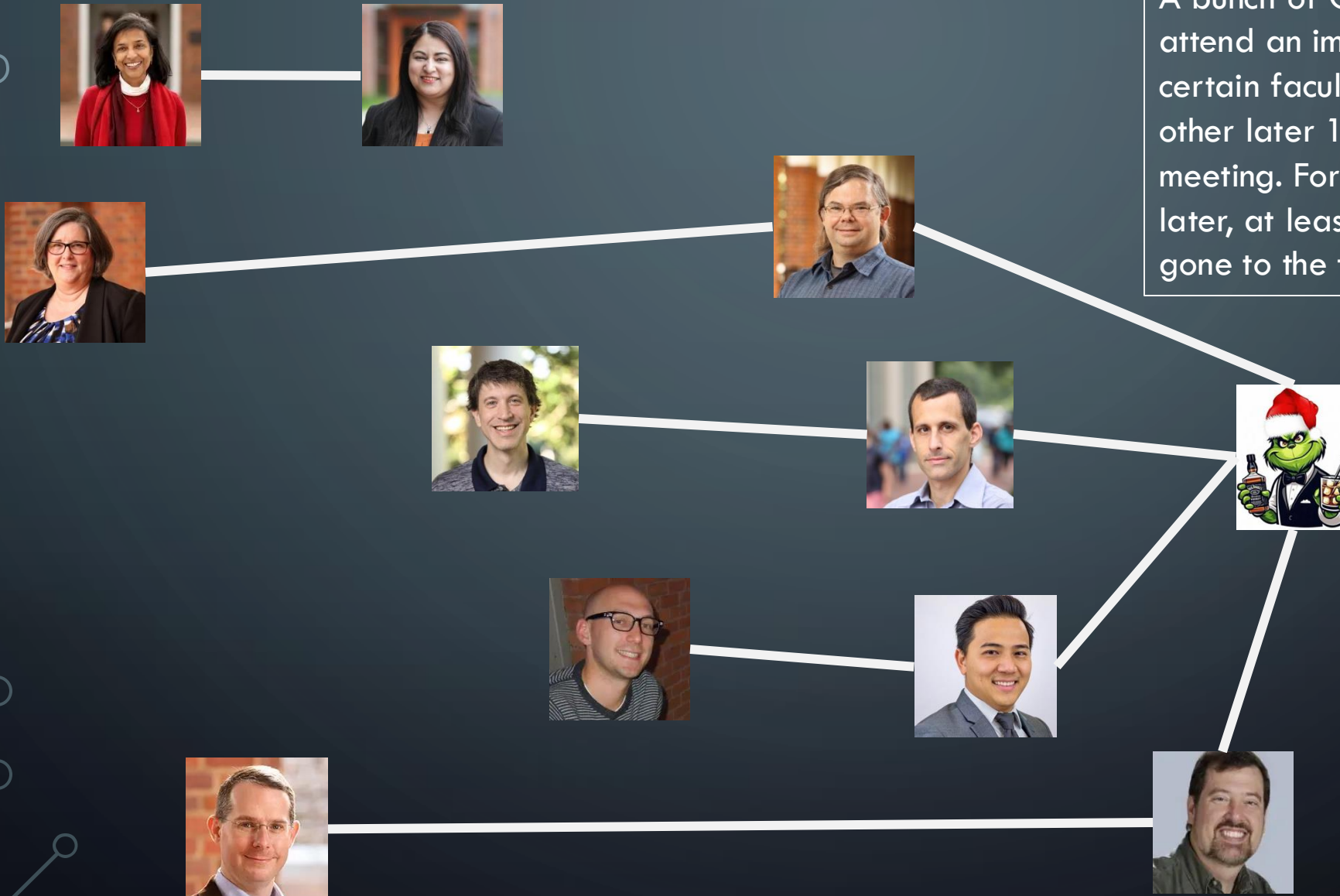


A bunch of CS faculty don't want to attend an important meeting...



PROBLEM B: MEETING NOTES

A bunch of CS faculty don't want to attend an important meeting...but certain faculty members will see each other later 1-on-1 to discuss the faculty meeting. For EVERY 1-on-1 meeting later, at least one of the two must have gone to the faculty meeting.



PROBLEM B: VERTEX COVER

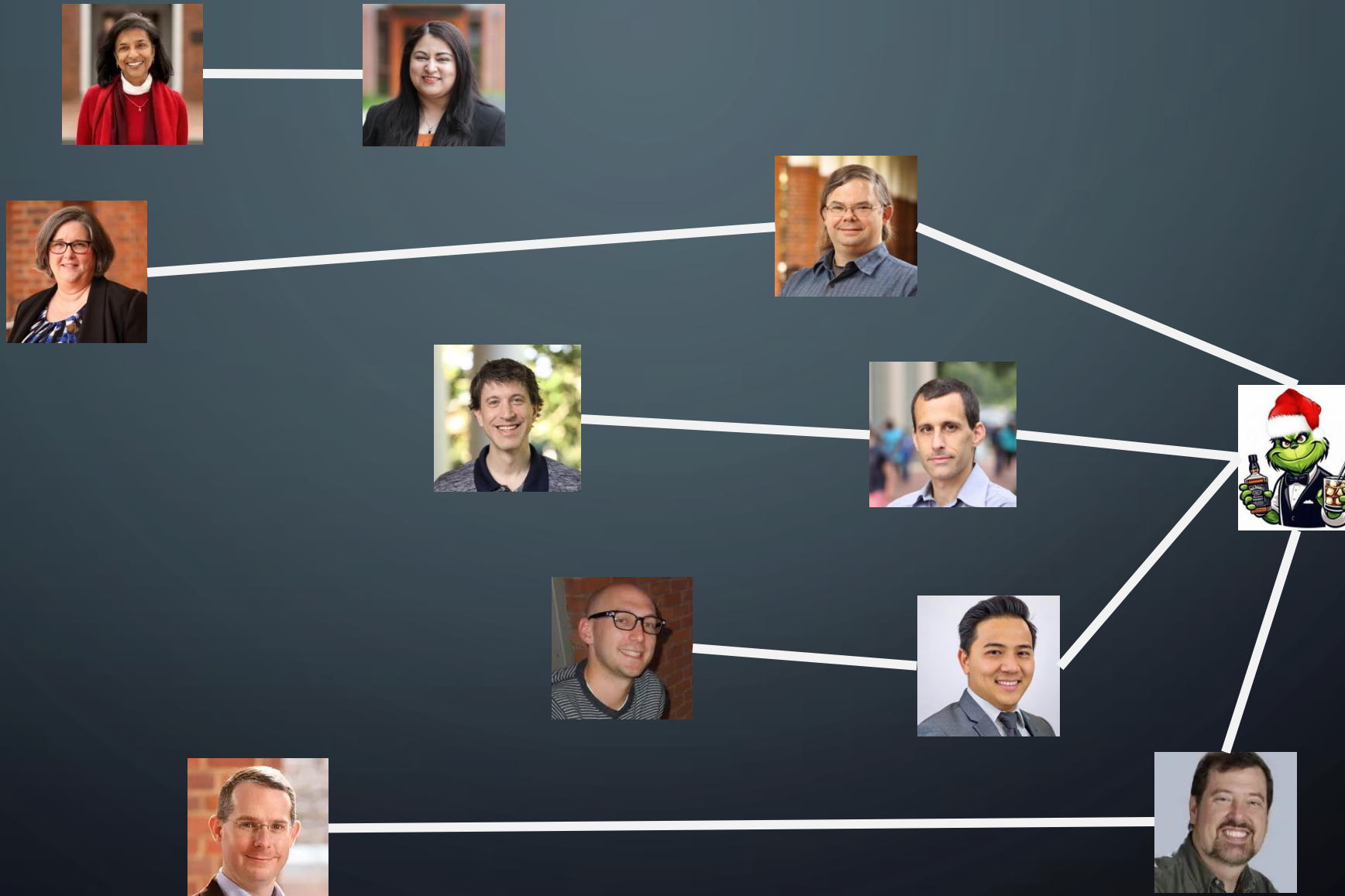
Vertex Cover: What is the minimum number of people that need to go if for every 1-on-1 meeting later in the day, at least one person must have been at the faculty meeting?



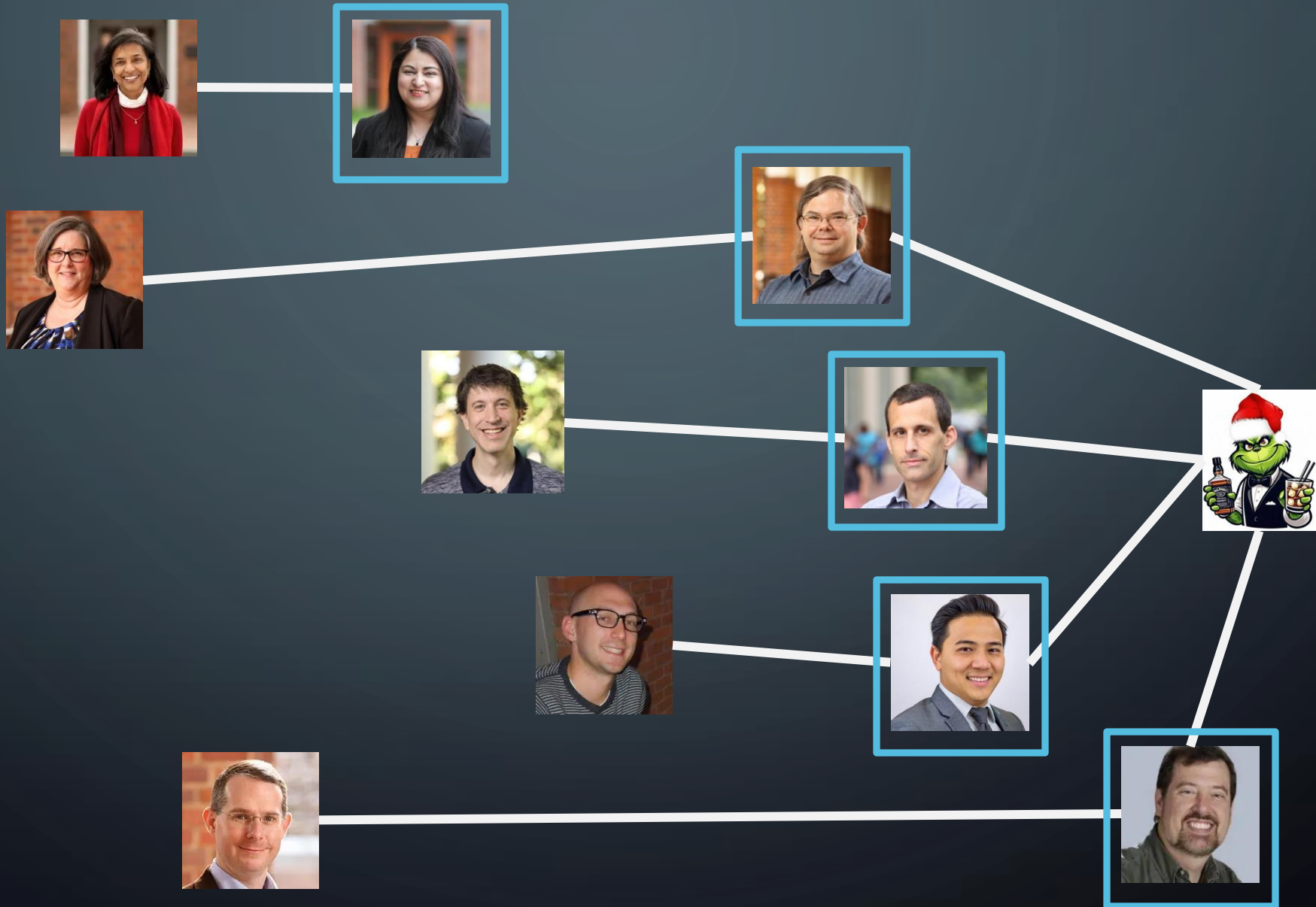
PROBLEM B: VERTEX COVER

Minimum Vertex Cover: Given a graph $G = (V, E)$, what is the minimum set of nodes in a subset $S \subseteq V$ such that for each edge $e = (u, v)$, either $u \in S$ or $v \in S$ (or both)

PROBLEM B: VERTEX COVER



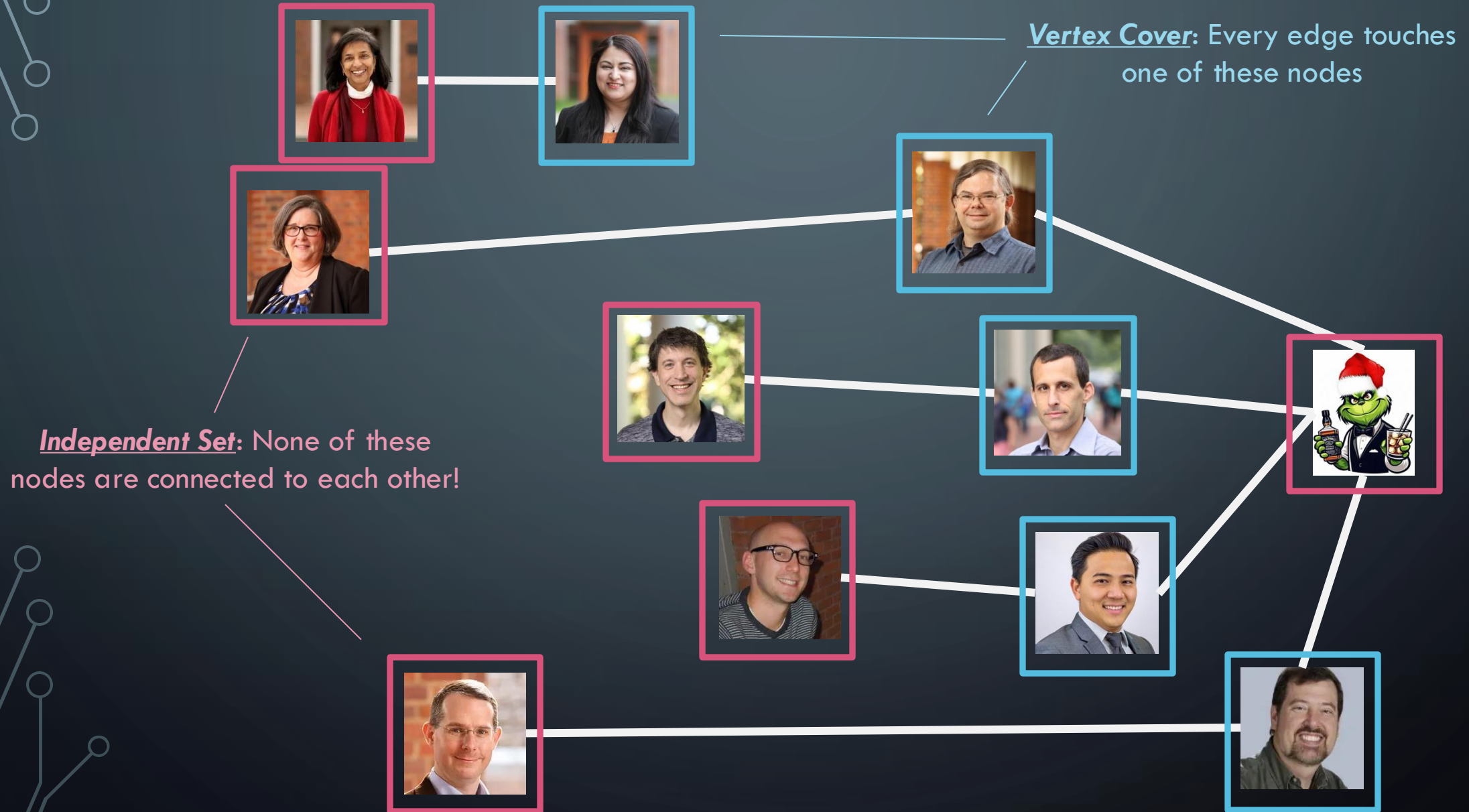
PROBLEM B: VERTEX COVER



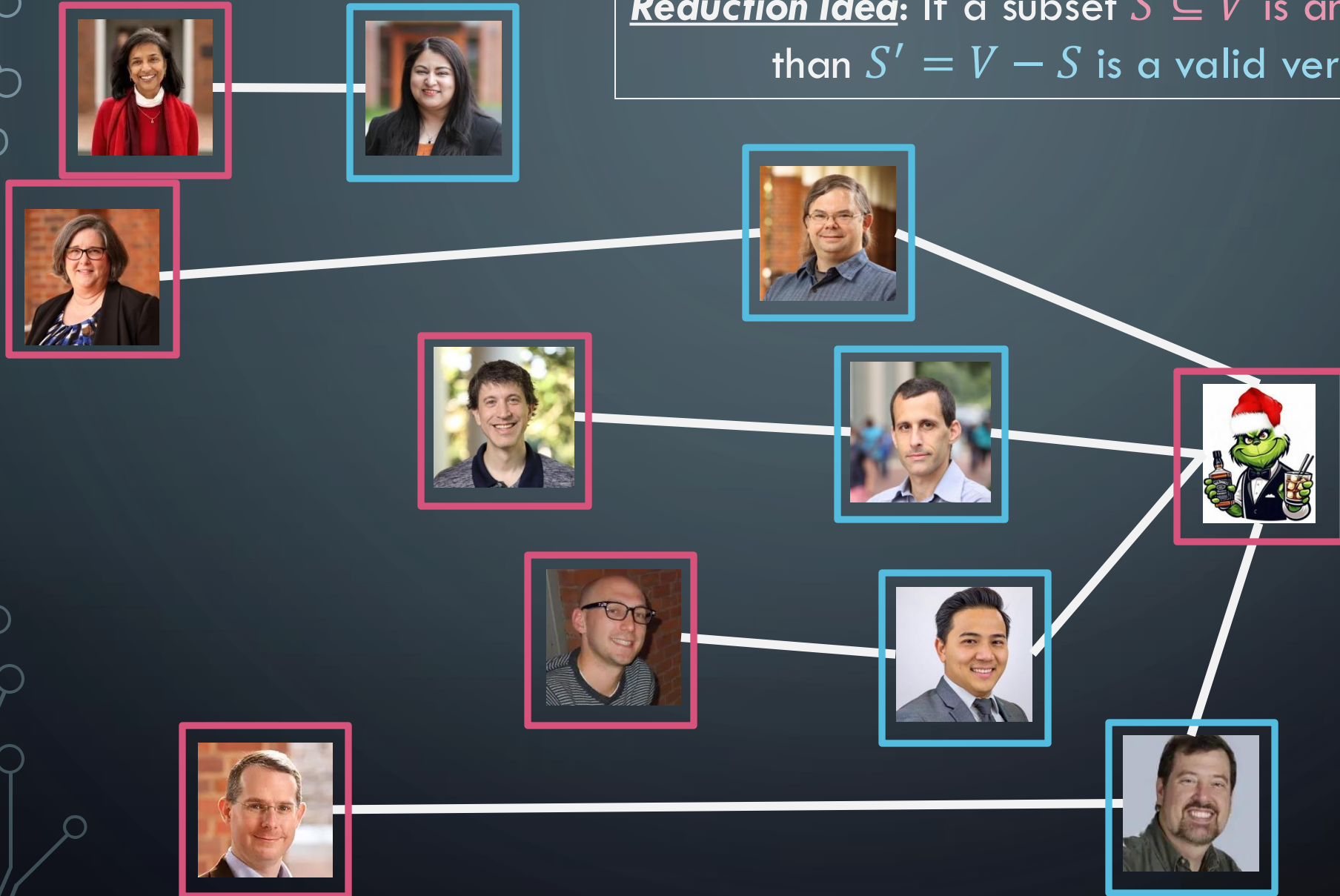
A decorative graphic on the left side of the slide consisting of a network of thin, light blue lines and small circles, resembling a circuit board or a neural network, extending vertically from the top to the bottom.

CONNECTING THE TWO PROBLEMS!

OBSERVATIONS



Reduction Idea: If a subset $S \subseteq V$ is an independent set,
then $S' = V - S$ is a valid vertex Cover.



Claim: If a subset $S \subseteq V$ is an independent set, then $S' = V - S$ is a valid vertex Cover.

Proof:

Assume $S \subseteq V$ is an independent set

Claim: If a subset $S \subseteq V$ is an independent set, then $S' = V - S$ is a valid vertex Cover.

Proof:

Assume $S \subseteq V$ is an independent set

Consider an arbitrary edge e in the graph

e

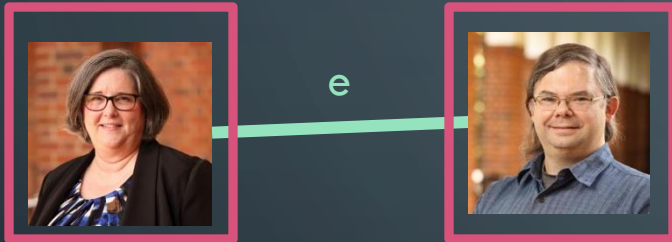
Claim: If a subset $S \subseteq V$ is an independent set, then $S' = V - S$ is a valid vertex Cover.

Proof:

Assume $S \subseteq V$ is an independent set

Consider an arbitrary edge e in the graph

It cannot be the case that both nodes are in S



*This is impossible because
then S would not be
independent set!*

Claim: If a subset $S \subseteq V$ is an independent set, then $S' = V - S$ is a valid vertex Cover.

Proof:

Assume $S \subseteq V$ is an independent set

Consider an arbitrary edge e in the graph

It cannot be the case that both nodes are in S

Thus at least one node is NOT in S



e



Claim: If a subset $S \subseteq V$ is an independent set, then $S' = V - S$ is a valid vertex Cover.

Proof:

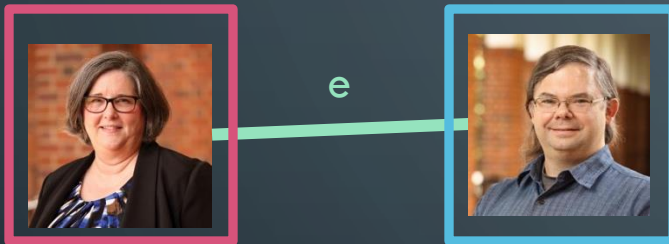
Assume $S \subseteq V$ is an independent set

Consider an arbitrary edge e in the graph

It cannot be the case that both nodes are in S

Thus at least one node is NOT in S

S' contains that node and covers the edge e



Claim: If a subset $S' = V - S$ is a valid vertex Cover,
then $S \subseteq V$ is an independent set

Proof:

Assume $S' \subseteq V$ is vertex cover

Claim: If a subset $S' = V - S$ is a valid vertex Cover,
then $S \subseteq V$ is an independent set

Proof:

Assume $S' \subseteq V$ is vertex cover

Consider an arbitrary edge e in the graph

e



e



Claim: If a subset $S' = V - S$ is a valid vertex Cover,
then $S \subseteq V$ is an independent set

Proof:

Assume $S' \subseteq V$ is vertex cover

Consider an arbitrary edge e in the graph

At least one (or both) nodes must be in S'

Claim: If a subset $S' = V - S$ is a valid vertex Cover, then $S \subseteq V$ is an independent set

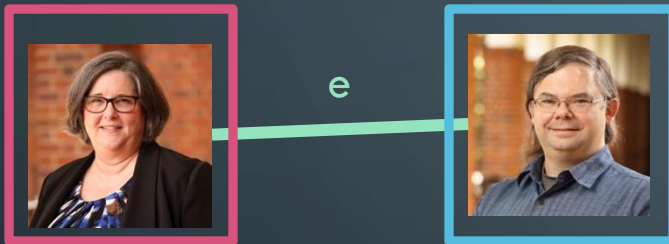
Proof:

Assume $S' \subseteq V$ is vertex cover

Consider an arbitrary edge e in the graph

At least one (or both) nodes must be in S'

Thus, $S = V - S'$ cannot contain both nodes



S will not contain both nodes, might contain one or zero

Claim: If a subset $S' = V - S$ is a valid vertex Cover, then $S \subseteq V$ is an independent set

Proof:

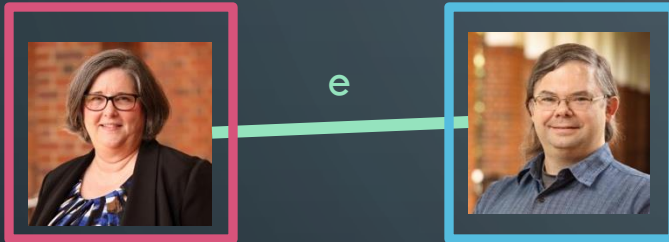
Assume $S' \subseteq V$ is vertex cover

Consider an arbitrary edge e in the graph

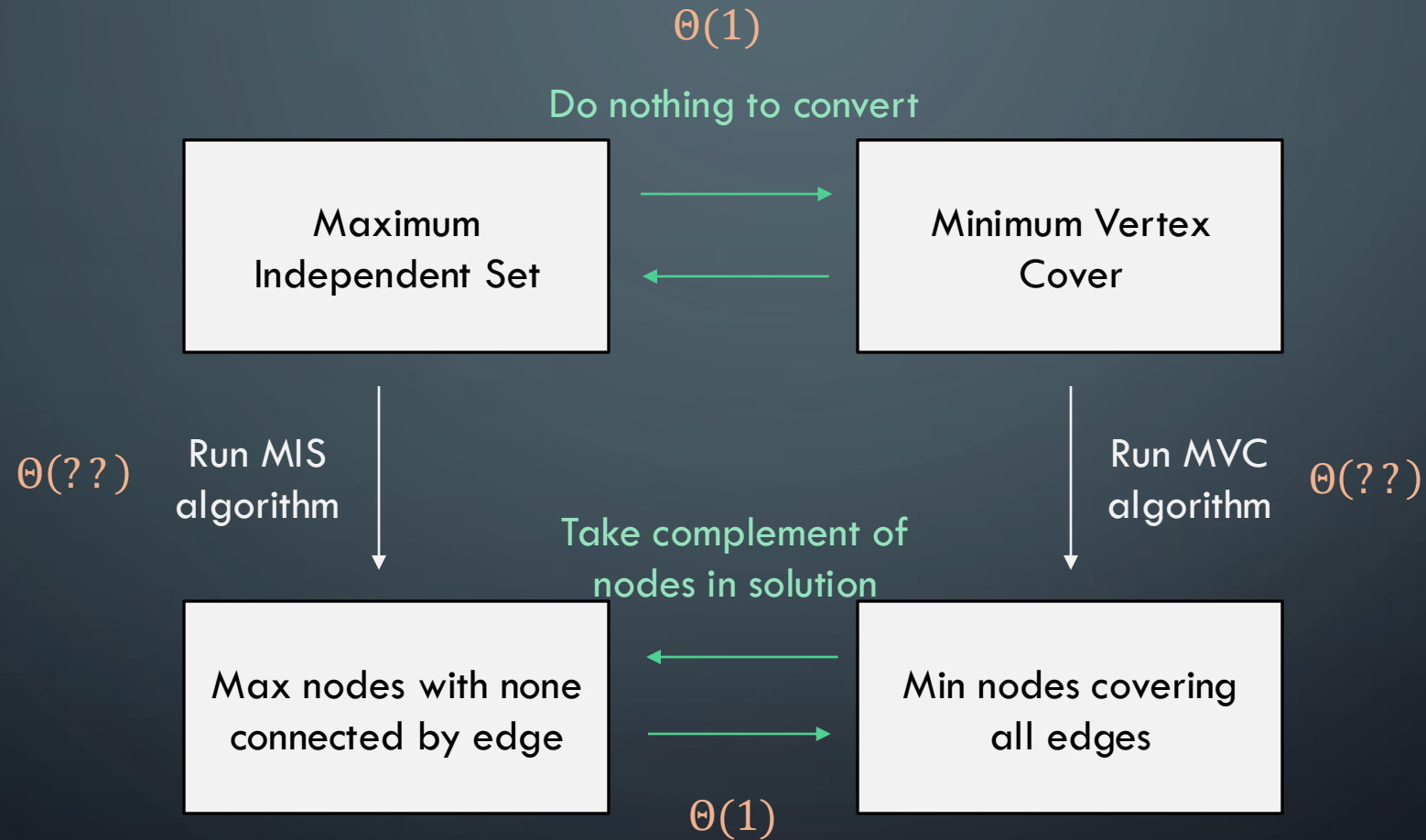
At least one (or both) nodes must be in S'

Thus, $S = S' - V$ cannot contain both nodes

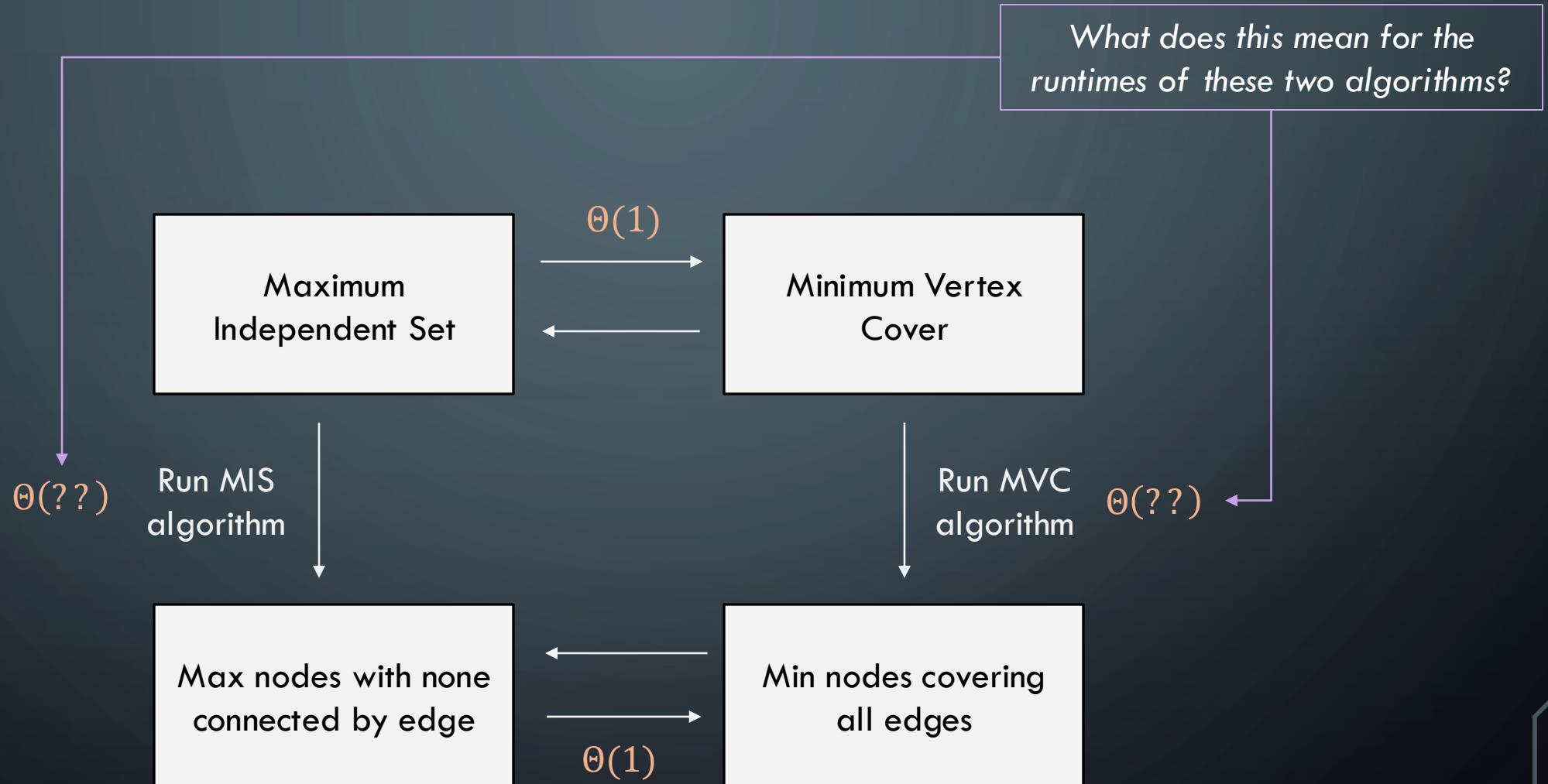
S does not have both nodes connecting the edge e and no conflict exists.



FINAL REDUCTION PICTURE



FINAL REDUCTION PICTURE



WHAT DID WE LEARN IN THIS SECTION

- Reductions!
 - Converting problems into another via reduction.
 - Use an old algorithm that works and only have to “develop” the conversion parts.
- Runtimes are linked!
 - Reductions establish a link between the runtimes of algorithms for the two problems. Sometimes the link goes in both directions!!